Mid Term review

Basics

What is a game ?

A game is an **activity** or sport involving skill, knowledge, and/or chance, in which you follow fixed rules and try to achieve a specific goal.

A **game** needs **goals** and **rules** (or mechanics)

Gameplay is the interactive aspects of video game design

- Gameplay involves player interaction with the game, usually for the purpose of entertainment, education or training.

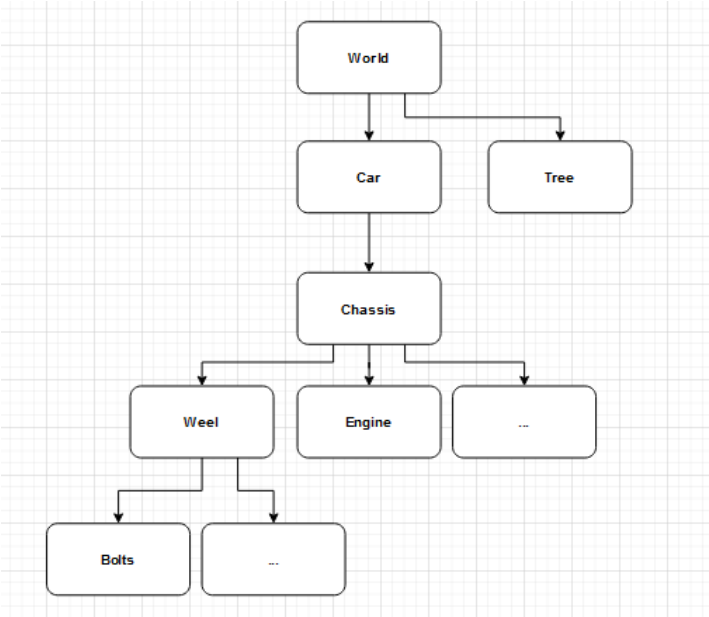
What is a game engine ?

A game engine is a **set of tools** used to simplify the process of making games. They usually include a list of functionality that are used in practically all games :

- A rendering engine ("renderer") for 2D or 3D graphics
- A physics engine for managing movement and collision detection ,
- A sound engine to manage sound in 2D or 3D space
- Some scripting tool to build interactivity
- An animation framework to allow the playing of animation
- An artificial intelligence framework to build NPC behaviours and make them navigate your levels
- Networking tools to build multiplayer games
- Memory management and monitoring tools
- A scene graph to manage objects in virtual space

What is a scene graph ?

Scene graph is **a structured representation** of a scene that can clearly express the objects**, attributes, and relationships between objects in the scene**.



Why use a scene graph ?

- It allows to create a **hierarchy** of objects
- Useful to define **local relationships** between objects instead of using a global relationship
- It allows to **group** objects and **move them relative to each other** (A car doesn't leave it's wheels behind when it drives off)

What is a game object ?

A node of the scene graph is called a **Game Object** in Unity

GameObjects are the **fundamental objects** in Unity that represent characters, props and scenery.

They do not accomplish much in themselves but they **act as containers** for **Components**, which implement the real functionality.

All game objects must have a **Transform** component

What is a transform ?

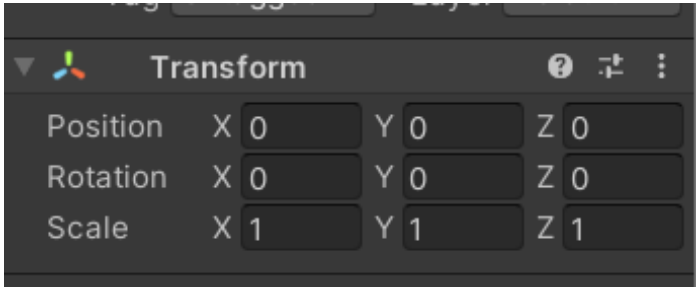
A transform is a component that contains the **position**, **rotation** and **scale** of an object.

Modifying the transform component of a gameobject will **change** the **position** , **rotation** or **scale** of that object.

A **transform** also contains others **vectors** that help described the position or direction of the game object such as the **forward** vector.

A transform in Unity

The transform component is define as such :



Global vs Relative transform

Global and Relative Transform

Global transformations describe where an item is located within the scene *relative to the origin*. The center of the game world, or the root of the scene graph

Local/Relative transformations describe how a gameobject is transformed ***relative to its direct parent node***. If you the GameObject is a **direct child** of the root its local transform and relative transform are the same.

A children-parent relation in the scene graph

Modifying the transform of parent node **will** modify the global transform of it’s child. And that applies to all it’s

But modifying the transform the child won't modify the parent node.

Components and Creating your own component

Why do we use a component based architecture ?

Components are the functional pieces of every **GameObject**. Components contain properties which you can edit to define the behavior of a GameObject.

They **can** be added at **runtime** but generally are added before during the edition process.

They are used to be able to customize the behaviour of a specific **GameObject**.

Monobehaviours

Every **public** variable will show up in the **inspector,** allowing you to modify **those value externally.**

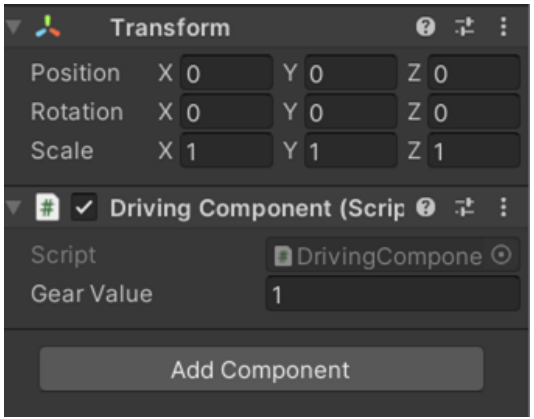
```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class DrivingComponent : MonoBehaviour
{
    public int GearValue = 1;
    // Start is called before the first frame update
    void Start()
    {

    }

    // Update is called once per frame
    void Update()
    {

    }
}
```



MonoBehaviour : Translate example

```
void Update()
{


    if (Input.GetKey(KeyCode.W))
    {
        transform.Translate(Vector3.up * Time.deltaTime);
        print("W key is held down");
    }

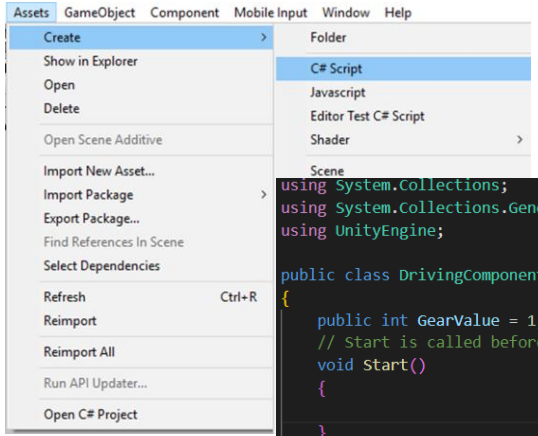
    if (Input.GetKey(KeyCode.S))
    {
        transform.Translate(Vector3.down * Time.deltaTime);
        print("S key is held down");
    }
}
```

Important class and components

Creating new components for your GameObject

Not all behaviour for our game already have existing components for the job. To add a new component we have to create a **script** that defines it's **behaviour.**

To add a new behaviour we need to create a new class that inherits a class **MonoBehaviour** in Unity




```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class DrivingComponent : MonoBehaviour
{
    public int GearValue = 1;
    // Start is called before the first frame update
    void Start()
    {

    }

    // Update is called once per frame
    void Update()
    {

    }
}
```

MonoBehaviours

By default, **when you create a new MonoBehaviours Unity** will ask you to define **2** methods:

- **Start** : A method that is called on every script to initialize them at the beginning of every game execution. Before the first frame update.

- **Update :** A method that is called once every frame.

Monobehaviours

Every **public** variable will show up in the **inspector,** allowing you to modify **those value externally.**

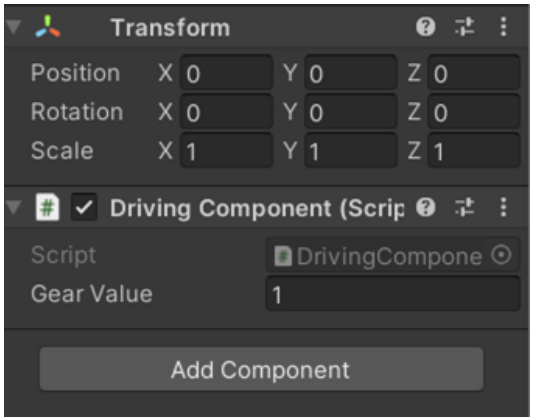
```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class DrivingComponent : MonoBehaviour
{
    public int GearValue = 1;
    // Start is called before the first frame update
    void Start()
    {

    }

    // Update is called once per frame
    void Update()
    {

    }
}
```



The different function of available in a MonoBehaviour

- The main function you will use :
  - Awake
    - Called when a Enabled Script is being loaded in the game. Before the Start() function.
  - Start()
    - Called before the first Update function.
  - Update()
    - Called every Frame. Is Frame-rate depedent
  - FixedUpdate()
    - Framerate independent is used for Physics calculation.
  - LateUpdate()
    - Is called after every GameObject has called their Update function. Useful if you want to make sure something was done prior to calling this logic.

- https://docs.unity3d.com/ScriptReference/MonoBehaviour.html

Unity Important class : Time

Unity's Time class provides important basic properties that allow you to work with time-related values in your project.

The Time class has a few properties which provide you with numeric values that allow you to measure time elapsing while your game or app is running. For example:

- `Time.time` returns the amount of time in seconds since your project started playing.
- `Time.deltaTime` returns the amount of time in seconds that elapsed since the last frame completed. This value varies depending on the **frames per second** (FPS) rate at which your game or app is running

Unity Important class : Time

The Time class also provides you with properties which allow you to control and limit how time elapses, for example:

- `Time.timeScale` controls the rate at which time elapses. You can read this value, or set it to control how fast time passes, allowing you to create slow-motion effects.
- `Time.fixedDeltaTime` controls the interval of Unity's **fixed timestep** loop (used for physics, and if you want to write deterministic time-based code).
- `Time.maximumDeltaTime` sets an upper limit on the amount of time the engine will report as having passed by the "delta time" properties above.

MonoBehaviours

- You can access the transform of a gameobject that contains your MonoBeaviour by using the “transform” shorthand.

- To move a transform it is advised to use the translate method.

Unity API : Camera component

Cameras are the devices that capture and display the world to the player. By customizing and manipulating cameras, you can make the presentation of your game truly unique. You can have an unlimited number of cameras in a **scene**

They can be set to render in any order, at any place on the screen, or only certain parts of the screen.

But without a camera your game will have nothing to display. So you need to have **at least one** camera in your scene.

Unity API: Point lights

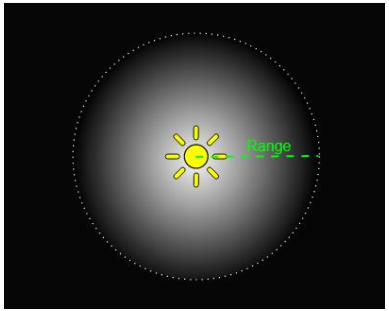
To be able to see in our game we use several types of lights to approximate lighting in the real world

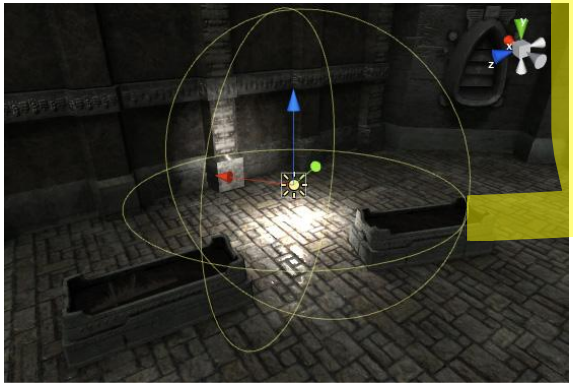
A **Point Light** is located at a point in space and sends light out in all directions **equally**. The intensity diminishes with **distance** from the light, reaching zero at a specified range.

Point lights are useful for simulating lamps and other local sources of light in a scene.

You can also use them to make a spark illuminate its surroundings.

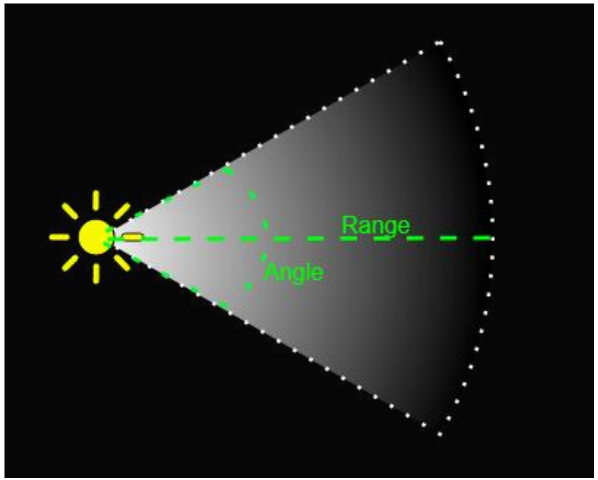
.

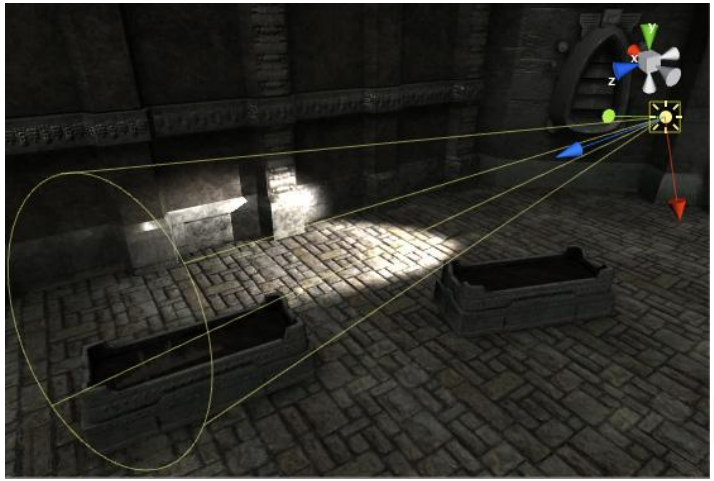




Unity API: Spot Lights

Like a Point Light, a **Spot Light** has a specified **location** and **range** over which the light falls off. However, a Spot Light is **constrained** to an **angle**, resulting in a **cone-shaped** region of illumination.

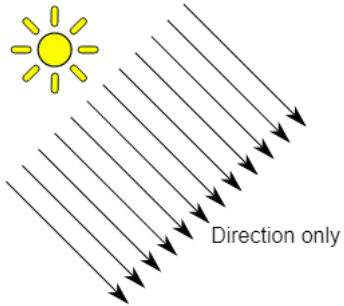


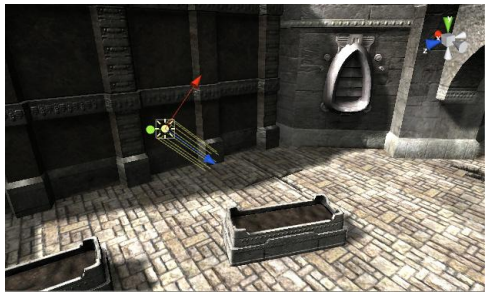


Unity API: Directional Lights

Directional Lights are useful for creating effects such as **sunlight** in your scenes. **Behaving** in many ways **like the sun**, directional lights can be thought of as **distant light sources** which exist very far away.

A Directional Light **doesn't have** any identifiable **source position** and so the light object **can be placed anywhere in the scene. All** objects in the scene are illuminated **as if the light is always from the same direction.** The **distance** of the light from the target object **isn't defined** and so the light **doesn't diminish**.





Meshes and Materials

Unity API : Meshes and Materials

To draw something in Unity, you must provide information that describes its shape, and information that describes the appearance of its surface.

You use **meshes** to describe **shapes**, and **materials** to describe the **appearance of surfaces.**

Materials are used in conjunction with **textures** to create complex artwork.

Input system

Unity API : Input

Interface into the Input system.

Use this class to read the axes set up in the Conventional Game Input, and to access multi-touch/accelerometer data on mobile devices.

The Input class has several methods to access the keys that are pressed as well as gamepad input.

Input flags **are not** reset until the next **Update**. You should make **all** the **Input** calls in the **Update Loop**.

Unity API : Input

Interface into the Input system.

Use this class to read the axes set up in the Conventional Game Input, and to access multi-touch/accelerometer data on mobile devices.

The Input class has several methods to access the keys that are pressed as well as gamepad input.

Input flags **are not** reset until the next **Update**. You should make **all** the **Input** calls in the **Update Loop**.

Unity API : Input

How to read inputs :

Input.GetKeyDown(KeyCode.Space)

will return true if the specified Keycode is **has been pressed down in the last frame**

bool value = Input.GetKeyUp(KeyCode.Space)

will return true if the specified Keycode **has been release in the last frame.**

bool value = Input.GetKey(KeyCode.Space)

will return true if the specified Keycode **is being currently pressed down.**

Unity API : Input

- Input.GetAxis(“Vertical”)
  will return the value of the vertical axis of the gamepad from (-1 to 1).
- Input.GetAxis(“Horizontal”)
  will return value of the horizontal axis of the gamepad from (-1 to 1).

Unity API : Input

bool value = Input.GetButtonDown(“Fire1”)

will return true if the button mapped to Fire1 has been **pressed down in the last frame**

bool value = Input.GetButtonUp(“Fire1”)

will return true if the button mapped to Fire1 **has been release in the last frame.**

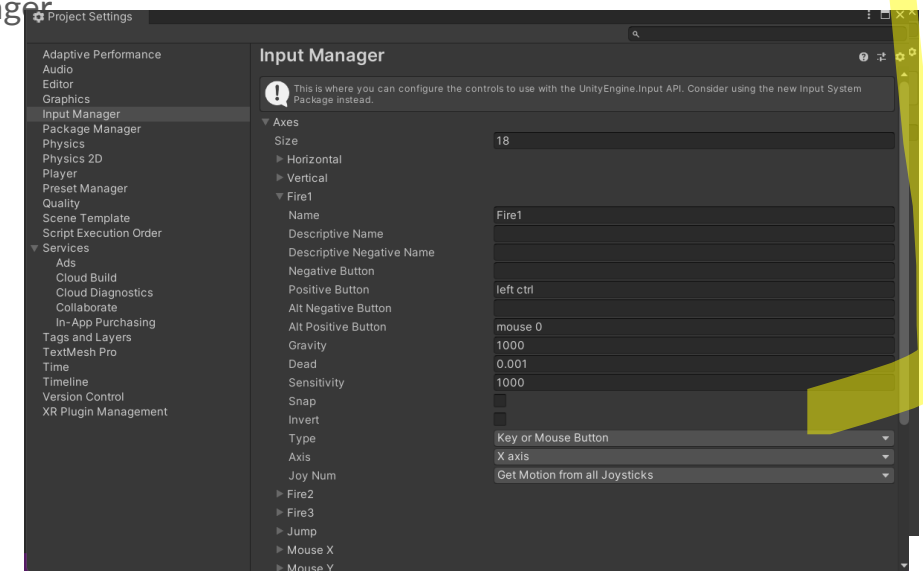
bool value = Input.GetButton(“Fire1”)

will return true if the button mapped to Fire1 **is currently being pressed down.**

Defining some Input button

Edit > Preferences > Project Settings > Input manager

Can be access with Input.GetButton (“Fire1”)



Unity API: Input (coding examples)

```
// Update is called once per frame
void Update()
{
    if (Input.GetKeyDown(KeyCode.W))
    {
        print("up arrow key is held down");
    }

    if (Input.GetKeyDown(KeyCode.S))
    {
        print("down arrow key is held down");
    }
}
```

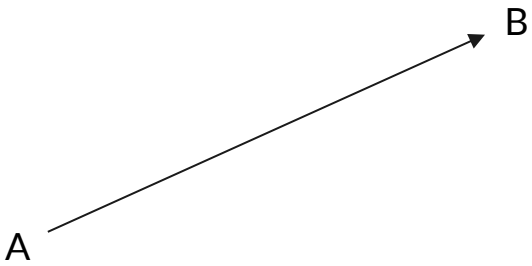
```
void Update()
{
    // Get the horizontal and vertical axis.
    // By default they are mapped to the arrow keys.
    // The value is in the range -1 to 1
    float translation = Input.GetAxis("Vertical") * speed;
    float rotation = Input.GetAxis("Horizontal") * rotationSpeed;
```

Linear algebra notion

Some notion of linear algebra

What is a **vector** ?

A vector is a quantity or phenomenon that has two independent properties: **magnitude** and **direction**. The term also denotes the mathematical or geometrical representation of such a quantity. Examples of vectors in nature are **velocity**, **momentum** and **force.**



Vectors in game development

In game development, vectors are mainly used to describe the **position** of a game object, or to determine its **speed** and **direction**.

In Unity 3D, vectors are an array of 3 values describing 3D coordinates so (x,y,z).

```
Vector3 demoVector = new Vector3(10, 15, 20);
```

In this example, demoVector is defined such as X = 10, Y = 15, Z = 20

Normalizing a vector

When normalized, a vector keeps the same direction but its length is 1.0.

Note that the current vector is unchanged and a new normalized vector is returned. If you want to normalize the current vector, use Normalize function.

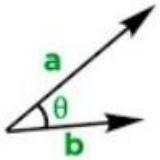
If the vector is too small to be normalized a zero vector will be returned.

```
Vector3 normalizedVector = Vector3.normalize(myVector);
```

Dot and Cross products : 2 very important linear algebra functions and how to use them in Unity.

Dot products

The dot product is a float value equal to the magnitudes of the two vectors multiplied together and then multiplied by the cosine of the angle between them.



$$a \cdot b = |a||b| \cos\theta$$

$$\cos\theta = \frac{a \cdot b}{|a||b|}$$

Dot product

The dot product measures how much two vectors point in the same direction.

If the result of the dot product of 2 normalize vector is:

Positive then the vectors point mostly in the same direction

Zero then the vectors are perpendicular (90° apart).

Negative then the vectors point mostly in opposite directions.

Dot products

You can use the dot product to determine the similarity of two vectors. The dot product between two vectors aiming in the same direction is 1:

```
var parallel = Vector3.Dot(Vector3.left, Vector3.left); // 1
```

The dot product between two vectors aiming in the opposite directions is also -1:

```
var opposite = Vector3.Dot(Vector3.left, Vector3.right); // -1
```

Dot products

And the dot product between two vectors at right angles to each other is 0:

```
var orthogonal = Vector3.Dot(Vector3.up, Vector3.forward); // 0
```

You can calculate the angle between the vectors by taking its arc cosine of the dot product.

```
var angle = Mathf.Acos(Vector3.Dot(Vector3.up, Vector3.forward));
```

Using the dot product to know if an object is front or behind.

The dot product is a great way to tell if an object is in front of or behind another.

To tell if one object is in front of another, we first need to decide what being in front of an object means. In Unity, the local z-axis represents the forward-facing direction, and you can access it through the forward property on an object's Transform.

We can **produce** a **vector** representing the **direction** from the **first object** to the **second** by **subtracting** the **position** of the **second** from the **position** of the **first**. We can then take the dot product of that vector against the forward direction of the first object.

```
Vector3 directionToOtherObject = someOtherObjectPosition - transform.position;
```

Using the dot product to know if an object is front or behind.

We can now use what we know about the dot product to figure out if the second object is in front of the first.

Recall that the dot product of two vectors aiming in the same direction is 1.

```
var directionToOtherObject =
    someOtherObjectPosition - transform.position;
var differenceFromMyForwardDirection =
    Vector3.Dot(transform.forward, directionToOtherObject);

if (differenceFromMyForwardDirection > 0)
{
    // The object is in front of us
}
else if (differenceFromMyForwardDirection < 0)
{
    // The object is behind us
}
else // differenceFromMyForwardDirection == 0
{
    // The object neither before or behind us; it's at a perfect
right
    // angle to our forward direction.( Can be left or right)
}
```

Script Lifecycle

Script lifecycle overview : First Scene load

These functions get called when a **scene** starts (once for each object in the scene).

Awake: This function is always called before any Start functions and also just after a **prefab** is instantiated. (If a GameObject is inactive during start up Awake is not called until it is made active.)

OnEnable: (only called if the Object is active): This function is called just after the object is enabled. This happens when a MonoBehaviour instance is created, such as when a level is loaded or a **GameObject** with the script component is instantiated.

Note that for objects added to the scene, the Awake and OnEnable functions for *all* **scripts** will be called before Start, Update, etc are called for any of them. Naturally, this cannot be enforced when you instantiate an object during gameplay.

Script lifecycle overview : Before the first frame update

This functions get called right before the first frame (once for each object in the scene):

Start: Start is called before the first frame update only if the script instance is enabled.

For objects that are part of a scene asset, the Start function is called on all scripts before Update, etc is called for any of them. Naturally, this cannot be enforced when you instantiate an object during gameplay.

Script lifecycle overview : Update order

The common pattern is to perform most tasks inside the **Update** function, but there are also other functions you can use.

FixedUpdate: FixedUpdate is often called more frequently than **Update**. It can be called multiple times per frame, if the frame rate is low and it may not be called between frames at all if the frame rate is high. All physics calculations and updates occur immediately after **FixedUpdate**. When applying movement calculations inside **FixedUpdate**, you do not need to multiply your values by **Time.deltaTime**. This is because **FixedUpdate** is called on a reliable timer, independent of the frame rate.

Update: Update is called once per frame. It is the main workhorse function for frame updates.

LateUpdate: LateUpdate is called once per frame, after **Update** has finished. Any calculations that are performed in **Update** will have completed when **LateUpdate** begins. A common use for **LateUpdate** would be a following third-person camera. If you make your character move and turn inside **Update**, you can perform all camera movement and rotation calculations in **LateUpdate**. This will ensure that the character has moved completely before the camera tracks its position.

Collisions and Physics

What are colliders

Collider **define the shape of an object for the purposes of physical collisions**. A collider, which is invisible, doesn’t need be the exact same shape as the object's mesh and in fact, **a rough approximation** is often more **efficient** and indistinguishable in gameplay.

What are colliders

The **simplest** (and **least processor-intensive**) **colliders** are primitive collider types. In 3D, these are the **Box Collider**, **Sphere Collider** and **Capsule Collider**.

A **capsule collider** is usually used for player collision because it allows smoother sliding motion in a incline.

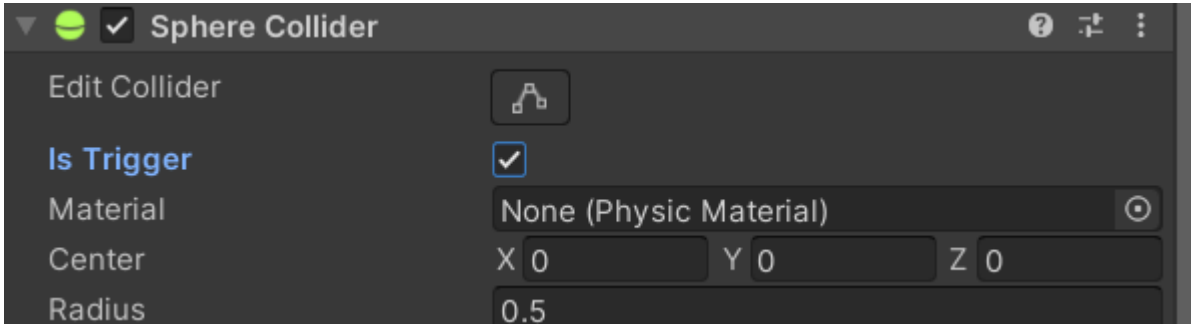
Any number of these can be added to a single object to create compound colliders. With careful positioning and sizing, compound colliders can often approximate the shape of an object quite well while keeping **a low processor overhead**. Simpler colliders are more efficient than bigger ones

Triggers vs Regular Colliders

What is a trigger ?

Triggers are a **special setup of colliders** that give them the ability to **trigger** events when they touch each other or overlap but that won't block object anymore. The object will simply pass but an event will be fired when an object is being overlapped by the trigger.

How to set it up :



How to deal with collision in code

For collision with dynamic objects we use the method OnCollisionEnter and OnCollisionExit

```
void OnCollisionEnter(Collision collision)
{
    foreach (ContactPoint contact in collision.contacts)
    {
        Debug.DrawRay(contact.point, contact.normal, Color.white);
    }
    if (collision.relativeVelocity.magnitude > 2)
        audioSource.Play();
}
```

How to deal with triggers

When were dealing with trigger since there is no collision we use the method OnTriggerEnter and OnTriggerExit to know when another object is overlapping the Trigger

```
//Upon collision with another GameObject, this GameObject will reverse direction
private void OnTriggerEnter(Collider other)
{
    speed = speed * -1;
}
```

Rigid body component

Rigid body

Rigidbodies are **components that allow a GameObject to react to real-time physics**. This includes reactions to **forces** and **gravity**, it defines **mass**, **drag** and **momentum**.

Since a Rigidbody component takes over the movement of the object it is attached to, you shouldn't try to move it from a script by changing the **Transform** properties such as **position** and **rotation**. Instead, you should apply **forces** to push the object and let the physics engine **calculate** the **results**

How to apply forces to a rigid body in code

```
void Start()
{
    //Fetch the Rigidbody from the GameObject with this script attached
    m_Rigidbody = GetComponent<Rigidbody>();
}

void FixedUpdate()
{
    if (Input.GetButton("Jump"))
    {
        //Apply a force to this Rigidbody in direction of this GameObjects up axis
        m_Rigidbody.AddForce(transform.up * m_Thrust);
    }
}
```

How to apply forces to a rigid body in code

What is a **FixedUpdate** ?

It is a method that is framerate independent is mainly used for physics update in the engine. It happens at a fixed time step meaning it happens at fixed time and is not affected by the frame rate. **It should always be used to deal with physical object or adding forces to object.**

It is important to know that the FixedUpdate is called **less often** in a second than Update() FixedUpdate is generally called 50 times a second and Update could be called upwards of 200 times a second a decent fps

Physics object types

Static vs Kinematic vs Dynamic

There exist 3 types of object in the physics engine

Static object : A static object is an object that doesn't move. They do not have a rigid body so even though the collider might be moved it won't generate collision events if moved.

Kinematic Object: Kinematic object are object that are considered to be moving but that their movement is not driven by the physics engine. Their movement is **entirely driven** by code. (IE Translate).

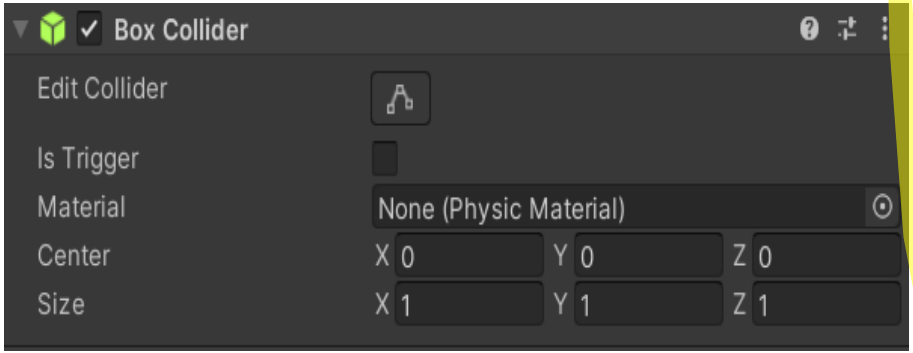
Dynamic Object : Dynamic object are object that move with the **physics engine**. They can be **pushed**, or move by applying forces on the object.

Static object

Static object **only** have a collider and no rigidbody components in Unity.

They are not considered to be moving by the physics engine. So dynamic object won't collide with it if it moved.

A static object will ignore all physic forces applied to it

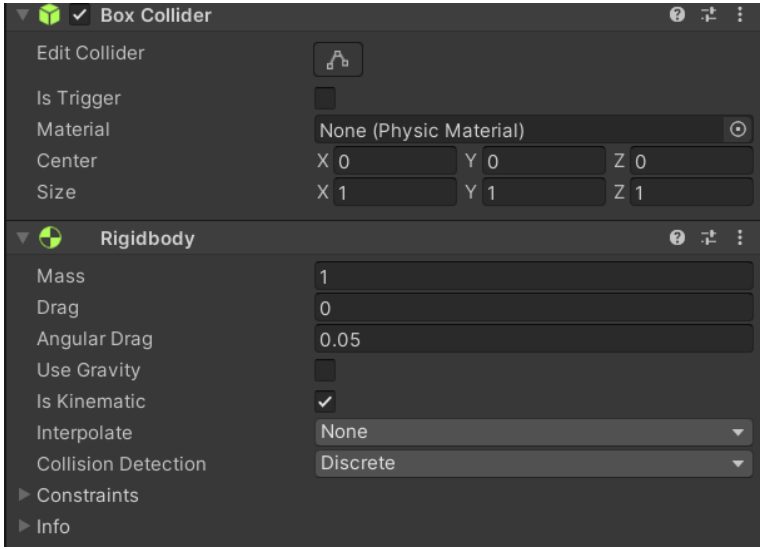


Kinematic object

To setup a kinematic object in Unity your object has to have both a **Rigid Body** and a **Collider** and to change the **Is Kinematic** option to **true**

The mass, drag option and use gravity will be ignored. **To move a kinematic object you need to modify his the transform.**

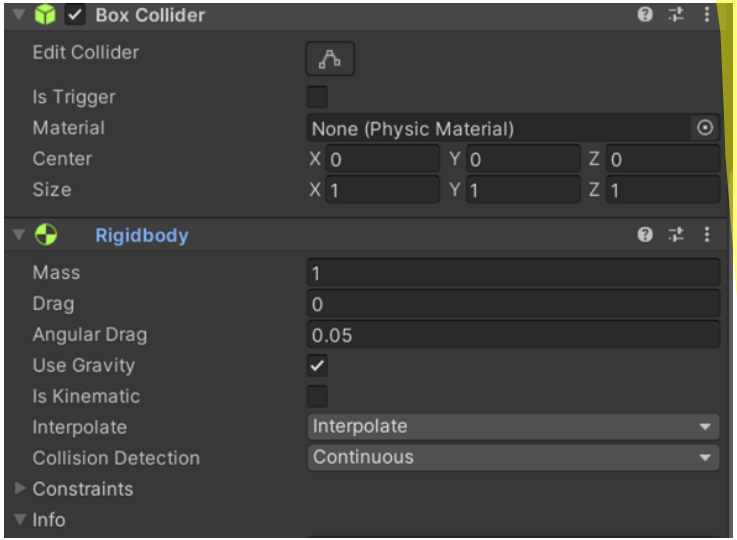
A kinematic object will ignore all physical forces applied to it but the collision is updated if you move it.



Dynamic object

To setup a dynamic object in Unity your object has to have both a **Rigid Body** and a **Collider and have the Is Kinematic set to false**

A dynamic object will collide with a static object, with kinematic object, as well other dynamic objects



Rotation

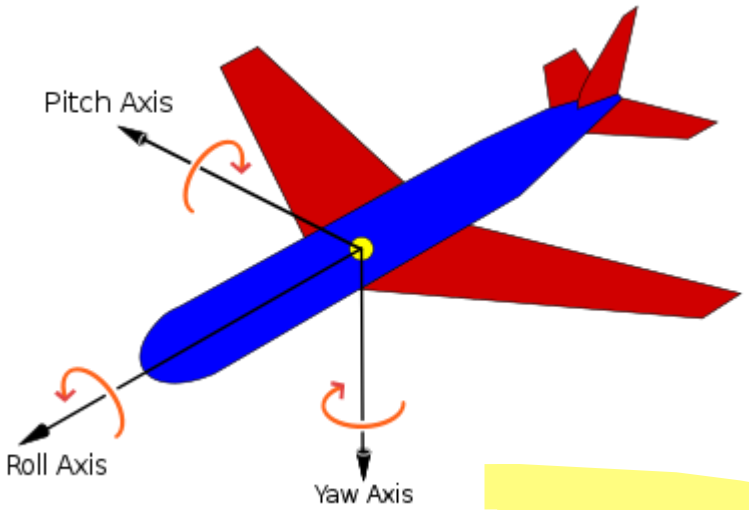
Euler angles

Pitch is rotation around the X axis

Roll is rotation around the Z axis

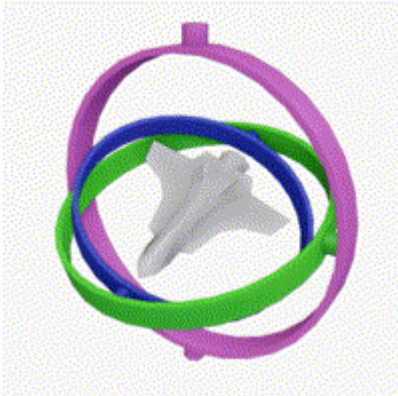
Yaw is a rotation around the Y axis

The order of rotation (usually XYZ) is important because rotating changes the object relative axis of rotation.



Gimbal lock

Gimbal lock is the **loss** of **one degree** of freedom in a three-dimensional, three-gimbal mechanism that occurs when the axes of two of the three gimbals are driven into a parallel configuration, "**locking**" the system into rotation in a degenerate two-dimensional space.



Quaternions

- In order to avoid Gimble locks we use a 4 parameter vector called a Quaternions.
- The purpose of the quaternion is to avoid gimble locks.

Prefabs and Instantiating objects

Prefabs

Prefabs are **a special type of component that allows fully configured GameObjects to be saved in the Project for reuse**. These assets can then be shared between scenes, or even other projects without having to be configured again. This is quite useful for objects that will be used many times, such as platforms.

Why use prefabs ?

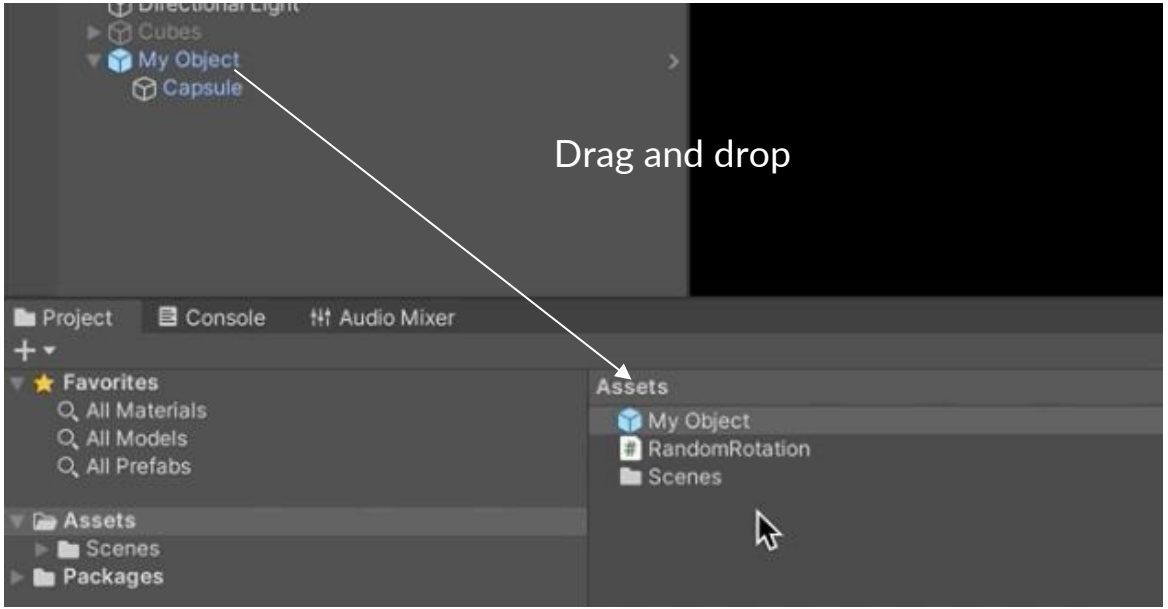
The prefab assets **allow you to store an asset with all its properties inside the prefab**, and the prefab acts basically as a template that you use to create new instances of the same object in the scene.

How to create a prefab

The simplest way to do this is to **build your object inside your scene**. Then, once you're done, **drag** the object to your project's assets to create a **prefab** of it.Your new prefab will appear in your project's assets, while the object in your scene **will turn blue**, **highlighting** the connection between the prefab template and the object in the hierarchy.

You'll then be able to **delete** the object from your **scene** and use the **prefab** in the Instantiate function.

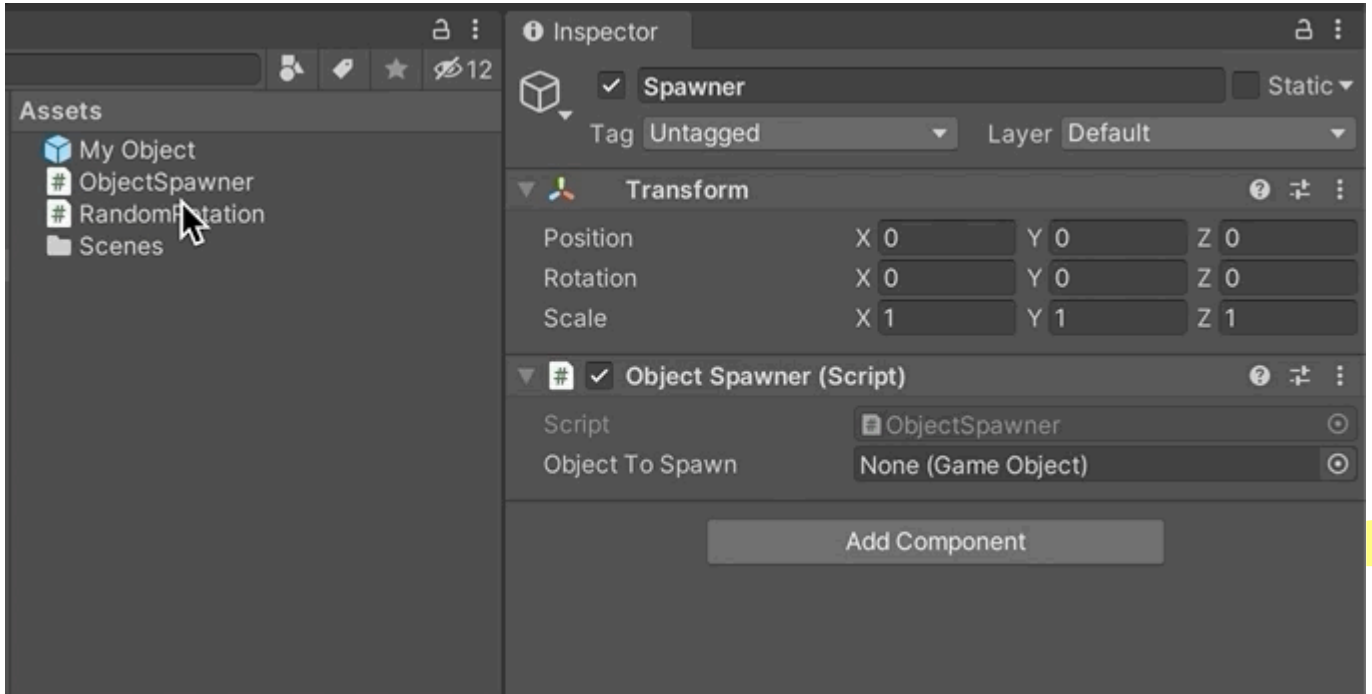
How to create a prefab



Spawning new GameObject via a Prefab

```
public GameObject objectToSpawn;
void Start()
{
    Instantiate(objectToSpawn);
}
```

Spawning a new GameObject via Prefab



Spawning a new GameObject with Prefab

To keep a reference to the spawn **GameObject** :

```
void SpawnOject()
{
    GameObject newObject = Instantiate(objectToSpawn);
}
```

How to set the rotation and position of a new object

While the basic overload method of Instantiate only requires a game object reference to work, chances are, you'll want to pass in more information than that when creating a new object.

By default, an **instantiated** object inherits the **position and rotation** of the object it's **cloned** from. Meaning that, if **you clone** an object in the scene, the new object will be created in the **same position as the original**.

How to set the rotation and position of a new object

The method to use to define Instantiate a GameObject with a specific location and a default rotation is :

```
Instantiate(objectToSpawn, transform.position, Quaternion.identity);
```

Tags

Finding different objects in the Scene

This will return a game object that matches an exact string.

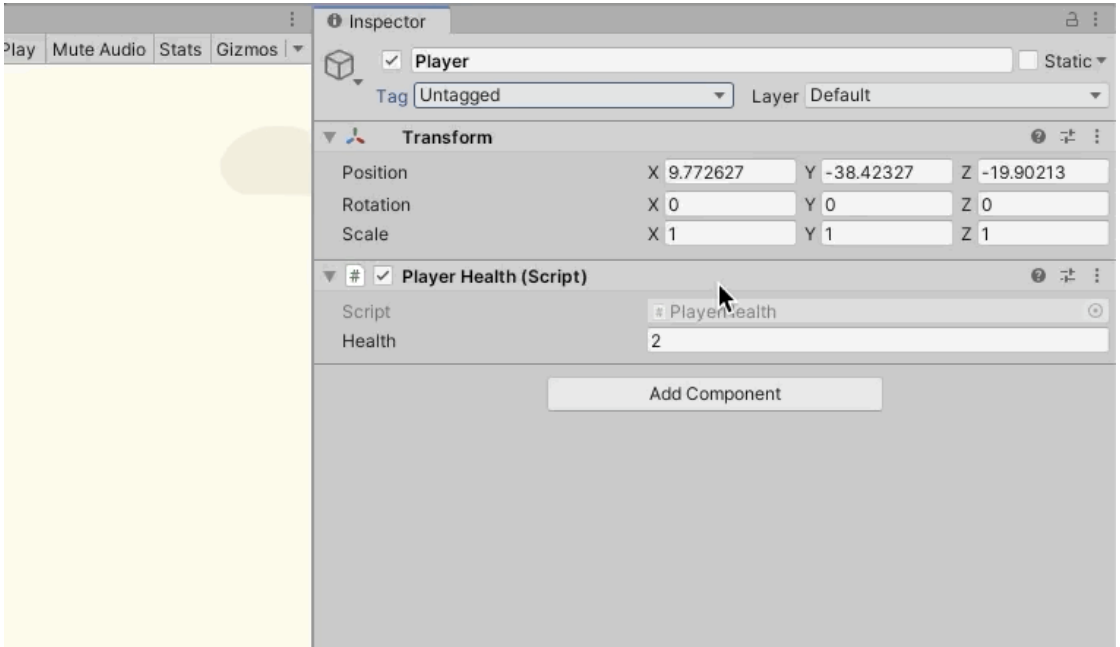
In this case, “Player”.

```
public class FindObject : MonoBehaviour
{
    public GameObject player;
    void Start()
    {
        player = GameObject.Find("Player");
    }
}
```

How to find an object by its tag in Unity

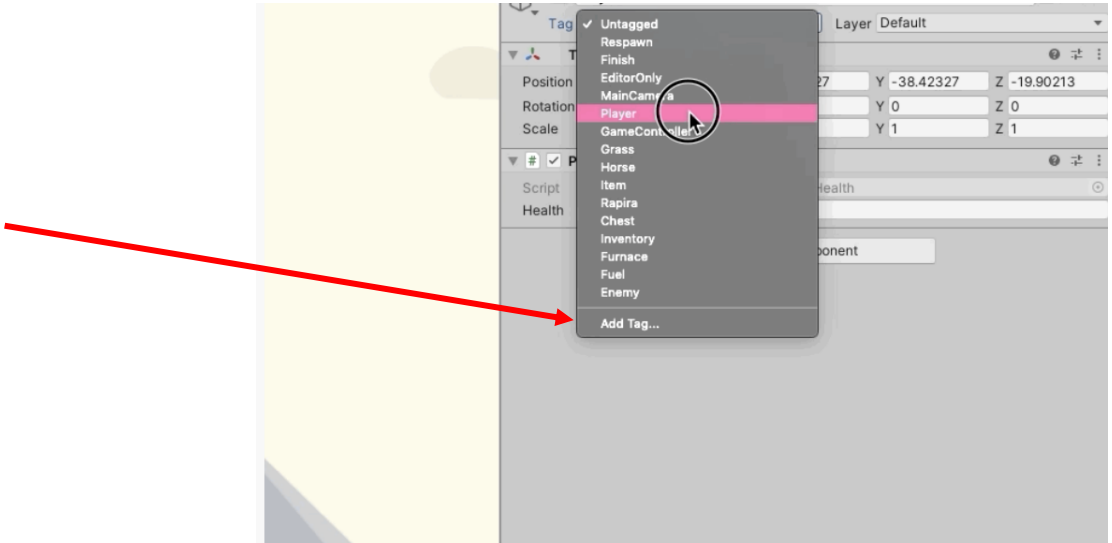
```
private void OnTriggerEnter(Collider other)
    {
        if (other.tag == "Player")
        {
            // Do Something
        }
    }
```

Setting a Tag



Adding new tags

In the Tag drop down at the very bottom is a **Add Tag** Option.



How to get multiple objects with tags

Just like Find with Tag, **Find Objects with Tag** finds objects in the Scene with a certain tag attached, adding them all to an array.

```
public class FindObject : MonoBehaviour
{
    public GameObject[] enemies;
    void Start()
    {
        enemies = GameObject.FindGameObjectsWithTag("Enemy");
    }
}
```

Global variables

Global variables in Unity

Global variables in Unity generally refer to a variable that has a single value, a single point of reference and that is accessible from any script.

There are several methods for creating **global variables in Unity**.

The simplest method for creating a global variable is to use a **static** variable.

Global variables Method 1: Static variables

The simplest method for creating a global variable is to use a **static** variable.

```
public static class PlayerHealth
{
    public static float health;
    public static float armour;
}
```

Global variables Method 1: Static variables

```
Float hp = PlayerHealth.health;
```

Global variables: Using statics to create a Singleton game manager

One method of using static variables to manage global references and values is to create a Singleton game manager.

A Singleton uses a **single**, **static**, point of **entry** via a **static instance reference.**

That way other scripts can access the Singleton, and its variables, without getting a reference to it first.

Global variables: Using statics to create a Singleton game manager

The **Game Manager** script then keeps a static reference to itself, which is accessible to other scripts.

This script **needs** to be added to **an object in the Scene** in order to work, so the class itself isn't **static**, just its **instance reference**.

Global variables : Singleton

```
public class GameManager : MonoBehaviour
{
    public static GameManager Instance { get; private set; }
    public float playerHealth=100;
    public float gameTime = 90;
    private void Awake()
    {
        Instance = this;
    }
}
```

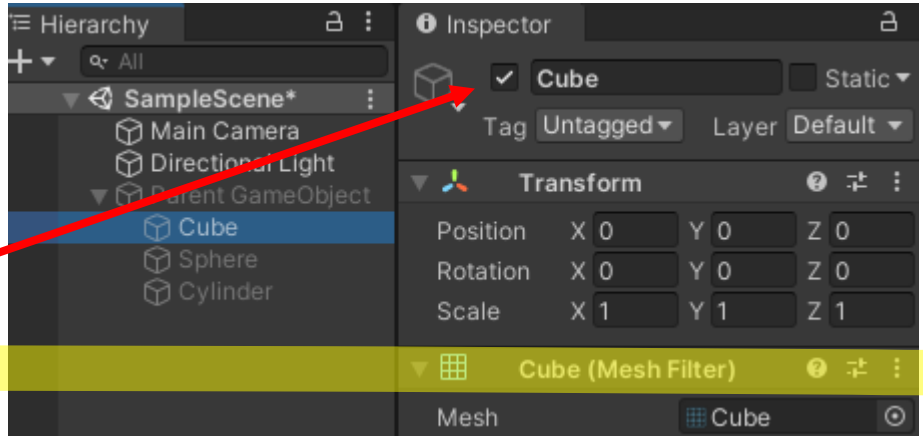
Accessing the singleton variables or methods

```
void Start()
    {
        Debug.Log(GameManager.Instance.playerHealth);
        Debug.Log(GameManager.Instance.gameTime);
    }
```

Destroying and Disabling object

Disabling GameObject

To **temporarily** remove a **GameObject** from your **scene** you can mark the GameObject as inactive.



How to disable in code ?

When disabling an object using the Set Active function, the game object will report itself to be disabled immediately.

```
gameObject.SetActive(false);
```

Destroying GameObject

To destroy an object in Unity, simply call the **Destroy** function, passing in a reference to the object that you want to destroy.

```
Destroy(Object objectToDestroy);
```

Delayed destroying of GameObject

It’s also possible to pass in a float delay value when calling the destroy method.

This will cause the destroy function to trigger in a number of seconds, which is specified by the float you pass in, instead of right away.

```
// Destroys the object in 4 seconds
Destroy(gameObject, 4);
```

When are destroyed objects removed in Unity?

It may surprise you to know that, when you destroy an object, **it isn't removed from the scene immediately.**

Instead, it's marked for deletion and removed after the current round of event functions that it's destroyed in.

For example, it's possible to call the destroy function on an object and then, immediately after "destroying" it, access the object in some way, without causing an error.

Destroying vs Deactivating

On Disable and **On Destroy** can be used to decommission an object before disabling it, either before turning it off or before it's destroyed.

They are call back to destroying an object or deactivating it but they do not happen at the same time.

Destroying vs Deactivating

```
void OnDisable()
{
    // Called when an object is turned off
}
void OnDestroy()
{
    // Called when an object is destroyed
}
```

Destroy vs Deactivating

When an object is destroyed, **On Disable** and **On Destroy** will both be called in its scripts, starting with On Disable, which is called at the same time as the initial Destroy function, and ending with On Destroy at the end of the frame.

Destroy vs Deactivating

However, while both destroying an object and simply turning it off will cause the On Disable function to be called, there is a slight difference between how the two methods work.

When disabling an object using the Set Active function, the game object will report itself to be disabled immediately.

What's more, even though it's turned off, other scripts with a reference to the object will be able to check if it is active or not for as long as the object still exists.

Should you destroy or disable an object?

Generally speaking, it's usually better to turn an object off than to destroy it, if you can.

Turning objects off instead of destroying them is the basic idea behind **Object Pooling**, which allows you to reuse objects over and over without having to recreate them, saving performance.

Should you destroy or disable an object ?

What's more, because disabled objects still, technically, exist, turning an object off can sometimes be easier to manage than destroying it.

For example, if a script in your scene still holds a reference to a disabled object, it's possible to check if that object is still active or not, by checking the **Active Self** or **Active in Hierarchy** properties.

How to know if an object is active ?

```
bool active = gameObject.activeSelf;
bool accessible = gameObject.activeInHierarchy;
```

Active Self will return true or false depending on if the object is, itself, turned on or off, while **Active in Hierarchy** only returns true if the object is active *and* accessible, as an active object can still be inaccessible if its parent is turned off.

Invoke, Coroutine and delayed execution

Using the invoke function

The purpose of the **Invoke** function is to call a method with a delay.

It works by passing in a **String** parameter, which is the name of the function that you want to call, and a **Float** delay, which is how long you want to wait before the function is triggered.

Using an Invoke

```
void Start()
{
    Invoke("MyFunction", 3);
}
void MyFunction()
{
    Debug.Log("Hello!");
}
```

Using an Invoke on another script

```
public OtherScript otherScript;
void Start()
{
    otherScript.Invoke("MyFunction", 5);
}
```

How to trigger a function regularly

Invoke Repeating works in the same way as Invoke, which uses a **String** to identify a function by name, and a **Float** to specify how long to wait before triggering it.

The difference is that Invoke Repeating will then, after the initial delay, repeat the function at a set interval.

Invoke Repeating

```
void Start()
{
    // Wait for 3 seconds then trigger My Function every 0.5 s
    InvokeRepeating("MyFunction", 3, 0.5f);
}
void MyFunction()
{
    // Do something!
}
```

How to cancel an already sent Invoke

You can cancel an already triggered Invoke function, before the method is executed, by using **Cancel Invoke**

:.

```
CancelInvoke("MyFunction");
```

```
CancelInvoke();
```

p your previously invoked function in this script , regardless of their

Drawbacks of Invoke

- The function is triggered by a string, so it’s possible to break the connection if its name is ever changed.
- A method triggered using Invoke can’t accept parameters.

What is a coroutine

A coroutine is a function that can be run over a number of frames, rather than all executed during a single frame.

Coroutine allows you to **pause** a function and tell it to **wait for a condition** or **action** to occur before continuing. This allows you to **split** up its functionality into a number of steps that can be executed in order.

How to use Coroutines ?

```
void Start()
{
    StartCoroutine(WaitForFunction());
}
IEnumerator WaitForFunction()
{
    yield return new WaitForSeconds(3);
    Debug.Log("Hello!");
}
```

Animations

Animation System Overview

Unity has a rich and sophisticated animation system (sometimes referred to as **'Mecanim'**)

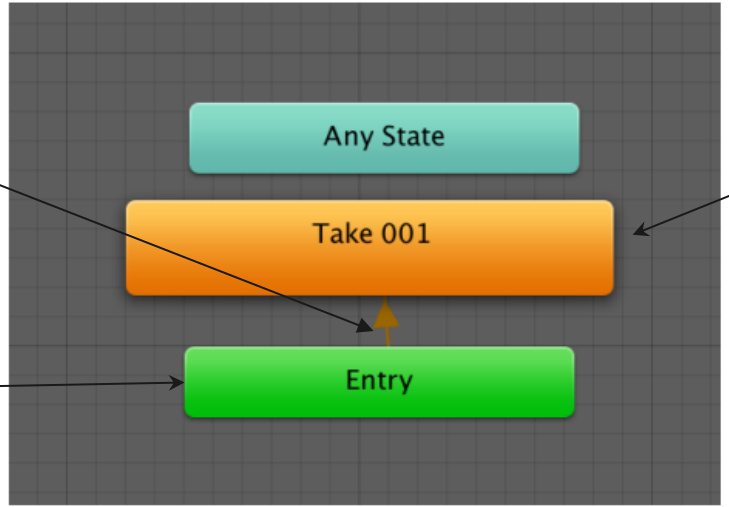
It provides:

- Easy workflow and setup of animations for all elements of Unity including objects, characters, and properties.
- Support for imported animation clips and animation created within Unity
- **Humanoid animation retargeting** (ability to apply animations from one character model onto another.)
- Layering and masking features

Animator Controller

Animated objects need an “**Animator Controller**” to actually play animations

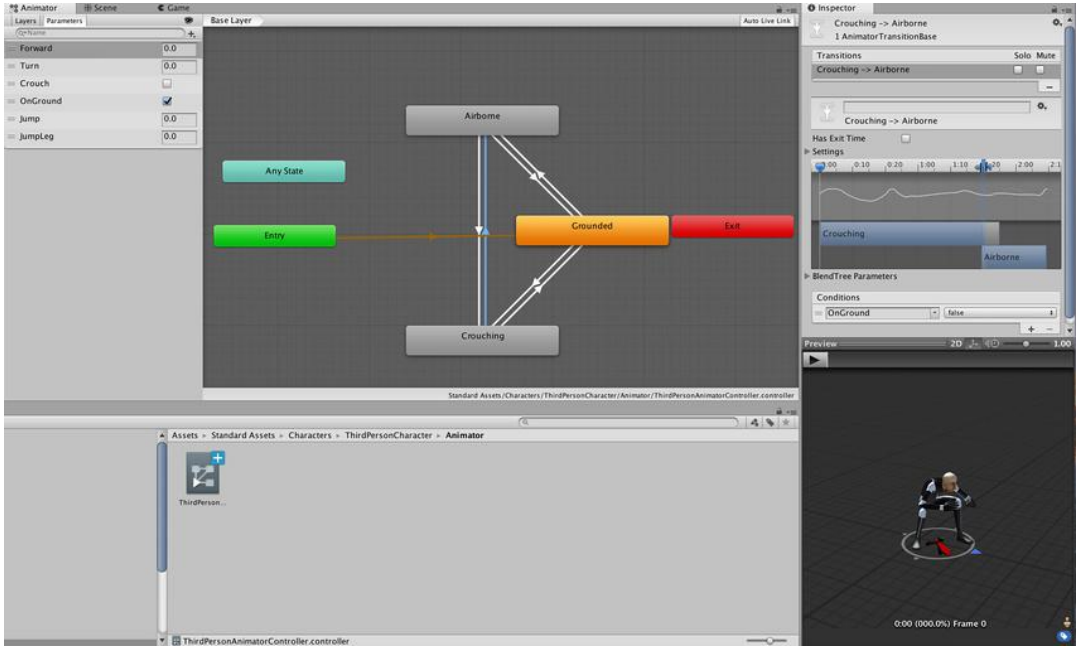
Transition (defines logic between animation states)



Default State
(Will run this animation if it can't run anything else)

Entry (Always Connected to Default State)

Animator Controller



How transition work ?

State transitions define how we can travel through the graph of animations.

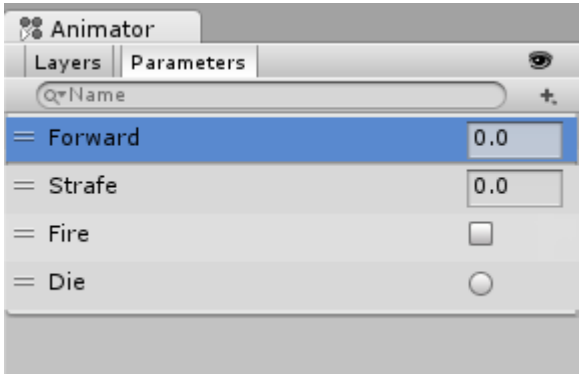
They are combined with **transition conditions** based on **Animator Controller parameters**;

Each transition has a set of **properties** we can adjust.

Click on the transition (white arrow) to select it. You can find the transition properties in the Inspector tab (make sure to unfold the Settings foldout):

Using parameters to control the animation flow

You can define a set of parameters in an **Animator Controller** and use them to control the transitions between animation states in Mecanim.



How to create new parameters and conditions

1. Open the Animator Controller asset.
2. Find the Parameters tab in the upper left corner of the Animator window and click on it.
3. Click on the plus icon in the Parameters tab to add a new parameter.
4. Choose a parameter type (integer,float,bool, trigger)
5. Type a name of the newly created parameter (in the provided example, the name of the parameter is Wave).

The Parameter types ;

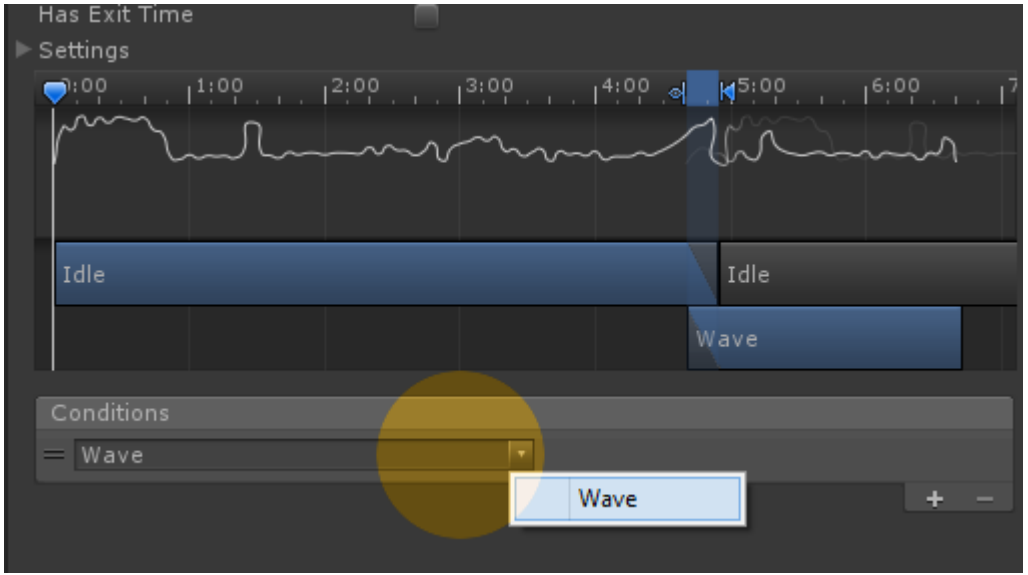
Integer - a whole number

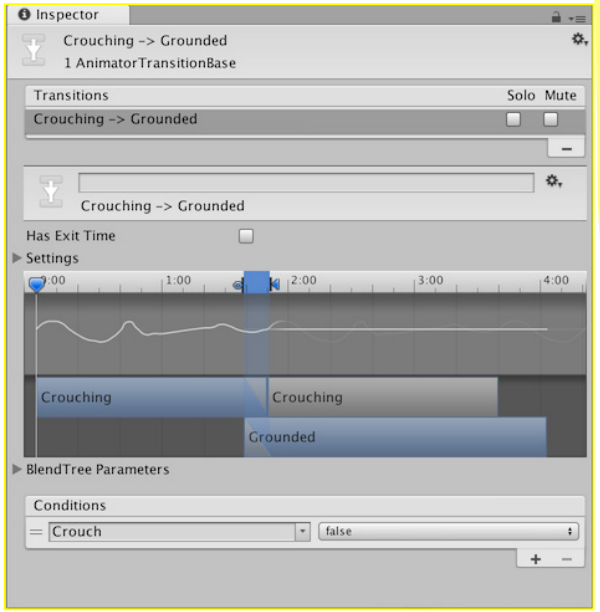
Float - a number with a fractional part

Bool - true or false value (represented by a checkbox)

Trigger - a boolean parameter that is reset by the controller when consumed by a transition (represented by a circle button)

Adding the condition to a Transition



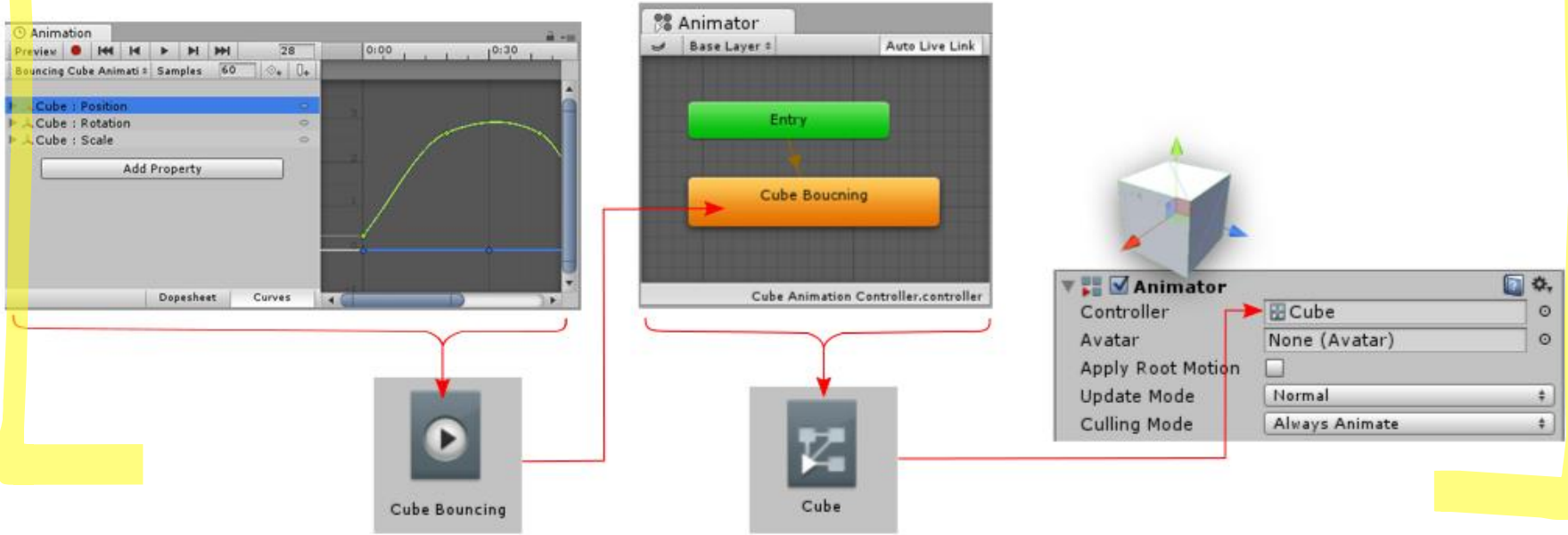


How it fits together

The above steps automatically set up the relevant components and references. However, it is useful to understand how the pieces connect.

- A GameObject must have an **Animator** component
- The Animator component must have an **Animator Controller**
- The Animator Controller asset must have one or more Animation Clips assigned

How it fits together



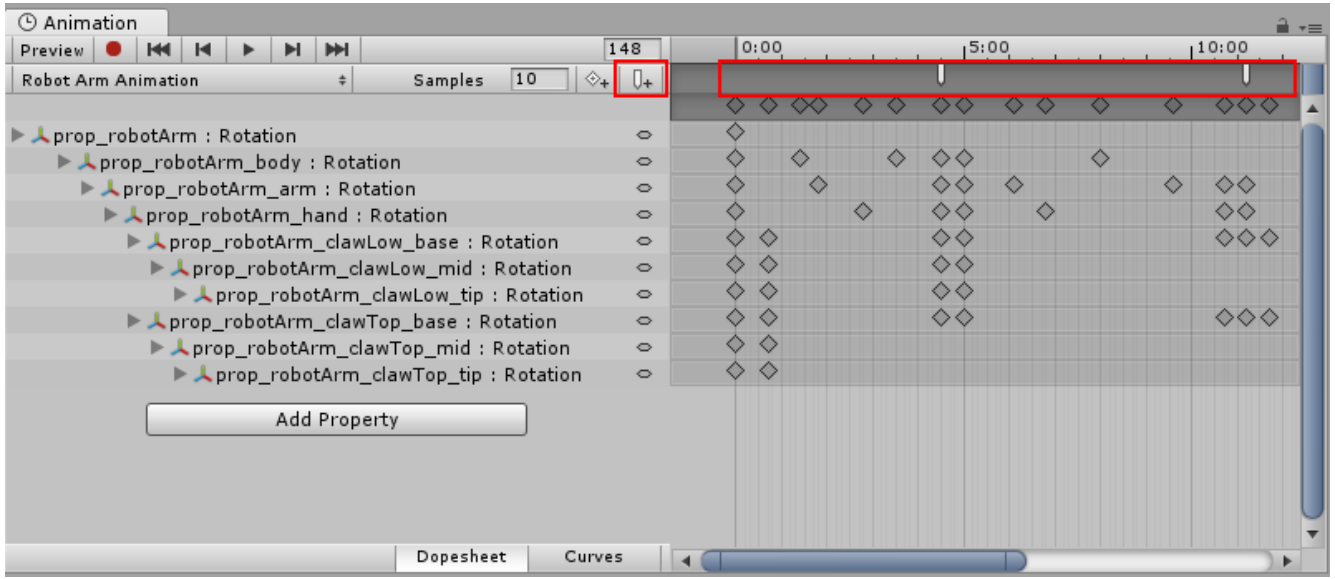
Animation events

An **Animation Event** is a special marker you can place on a specific frame of an animation in Unity (inside the Animation window) to trigger a function in a script when the animation reaches that point.

How it works

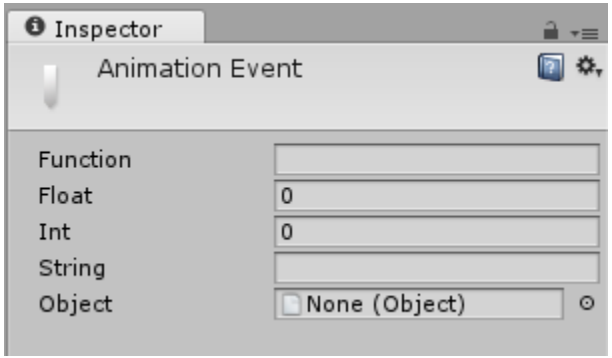
- When the animation plays and reaches the frame containing the event:
- Unity automatically calls a **public method** on the GameObject that has the same Animator.
- The function name must match the event's name.
- You can optionally pass **one parameter** (string, float, int, or object reference).

Animation Events



Animation events

When you have added an Event, the Inspector window will displays several fields. These fields allow you to specify the name of the function you want to call, and the value of the parameter you want to pass to it. As well as the specific object the that we have setup to catch the event.



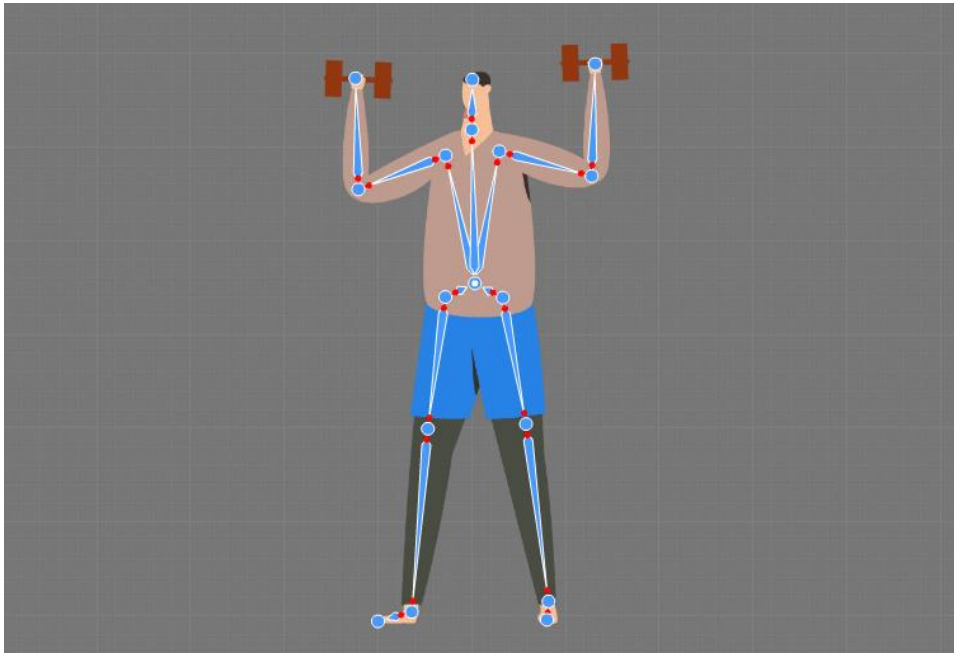
Skeletal Animation

Skeletal animation

Skeletal animation or **rigging** is a technique in computer animation in which a character (or other articulated object) is represented in two parts: a surface representation used to draw the character (called the **mesh** or **skin**) and a hierarchical set of interconnected parts (called **bones**, and collectively forming the **skeleton** or **rig**), a virtual armature used to animate (pose and keyframe) the mesh.

Skeletal animation

A 2D exemple :



Types of animation in Unity

The Animation System works with two types of models:

- A **Humanoid** model is a specific structure, containing at least 15 bones organized in a way that loosely conforms to an actual human skeleton
- A **Generic** model is everything else. This might be anything from a teakettle to a dragon

Blend trees

Blend trees

A common task in game animation is to blend between two or more similar motions. Perhaps the best known example is the blending of walking and running animations according to the character's speed.

Another example is a character leaning to the left or right as it turns during a run.

Blend trees

It is important to distinguish between Transitions and Blend Trees. While both are used for creating smooth animation, they are used for different kinds of situations.

- **Transitions** are used for transitioning smoothly from one Animation State to another over a given amount of time. Transitions are specified as part of an Animation State Machine . A transition from one motion to a completely different motion is usually fine if the transition is quick.
- **Blend Trees** are used for allowing multiple animations to be blended smoothly by incorporating parts of them all to varying degrees. The amount that each of the motions contributes to the final effect is controlled using a *blending parameter*, which is just one of the **numeric animation parameters** associated with the **Animator Controller**. In order for the blended motion to make sense, the motions that are blended **must be of similar nature and timing**. Blend Trees are a **special type of state** in an Animation State Machine

Types of blend trees

There are 3 types of blend trees

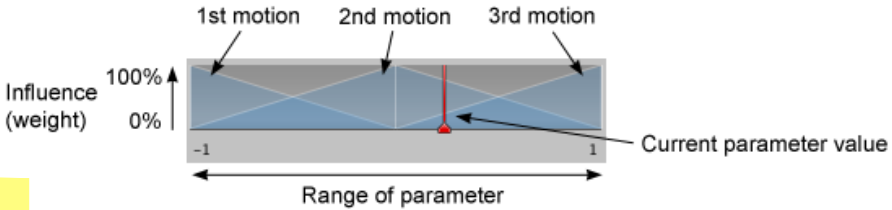
1. 1D Blend trees
2. 2D Blend trees
3. Direct Blend trees

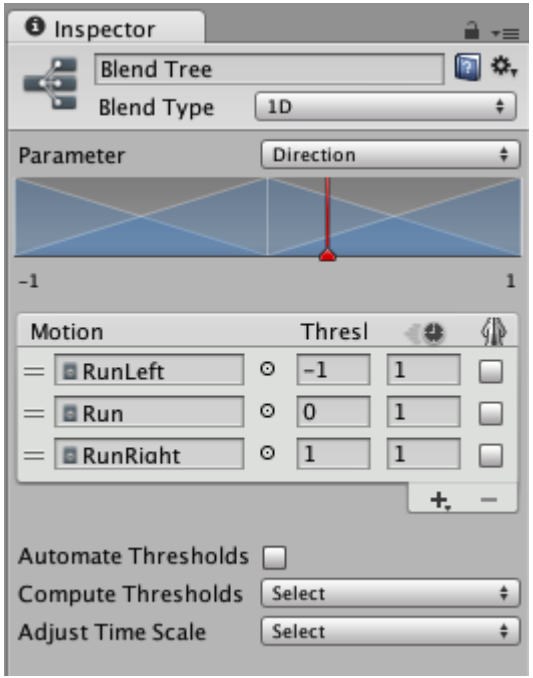
1D blending

1D Blending blends the motions of your two animation clips according to a single parameter.

The diagram at the top of the Inspector shows the influence of each of the child motions as the parameter varies between its minimum and maximum values.

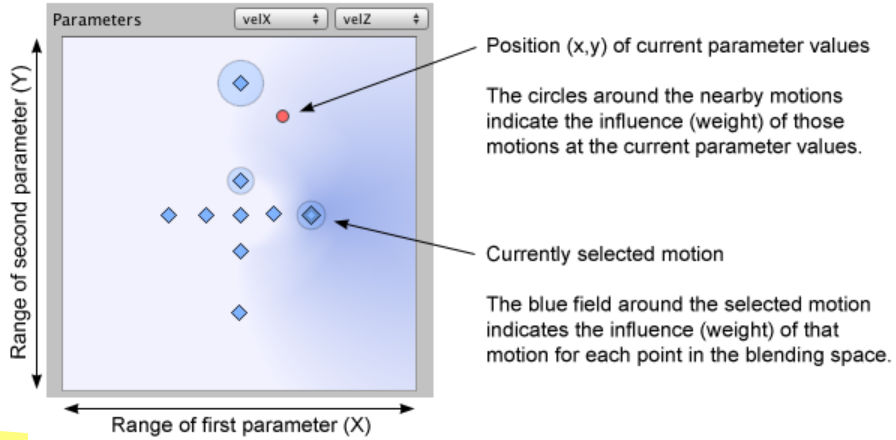
Each motion is shown as a little blue pyramid (the first and last are only shown in half), and if you click and hold down the left mouse button on one them, the corresponding motion is highlighted in the motion list below. The peak of each pyramid defines the parameter value where the motion has full influence, meaning that its animation weight is 1 and the other animations have a weight of 0. This is also called the

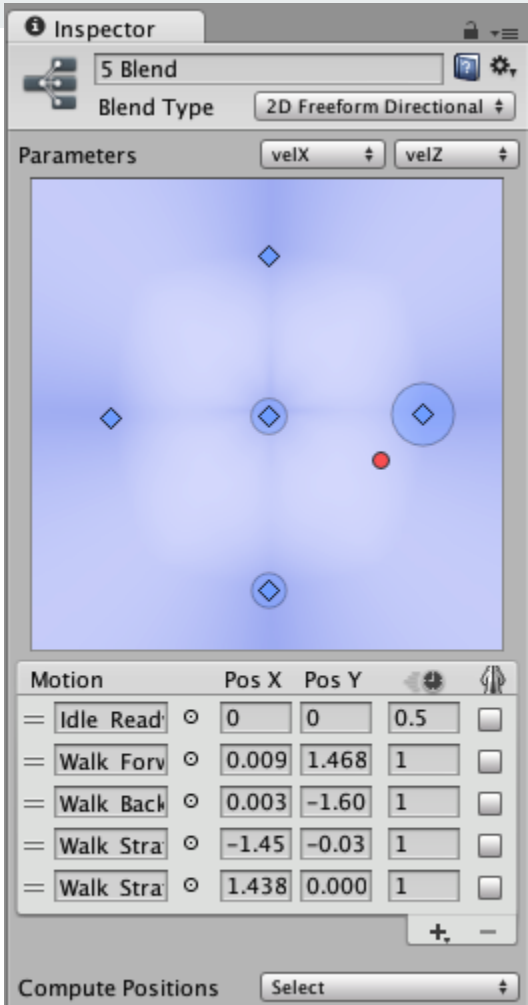




2D Blend trees

The 2D blending types blends the child motions according to two parameters.





The different systems in work in animation

. Root Motion

- Root motion means the character’s movement is driven directly by the animation itself (e.g., walking , running, jumping clips contain actual movement in world space). This ensures the motion feels natural and consistent with the animation, but requires careful setup to blend properly with gameplay logic.

Root motion

What it is: Instead of moving the character with code, the movement comes straight from the animation clip.

Example in Unity: A rolling animation actually “pushes” the character forward in the scene because the root of the character’s skeleton moves.

Why it’s useful: Keeps the character’s movement perfectly synced with the animation (no “sliding feet”).

Where it’s used: Games where animations must look very precise, like combat or cinematic sequences.

Blend spaces

Blend Space A blend space is a system that smoothly transitions between multiple animations based on input parameters (like speed or direction). For example, walking, jogging, and running animations can blend together so the character moves fluidly without hard switches.

Blend spaces

What it is: A system that blends between multiple animations depending on parameters (like speed, direction, or aiming angle).
Example in Unity: If the player's speed is 0 → idle animation. If speed is 3 → walk animation. If speed is 6 → run animation. Blend space will mix them smoothly, so the character doesn't suddenly "snap" from walk to run.
Why it's useful: Makes transitions smooth and natural.
Where it's used: Player locomotion, aiming systems, or any character with many possible movement states.

Inverse Kinematics (IK)

- Inverse Kinematics adjusts specific body parts (like hands, feet, or head) dynamically at runtime to match the environment or targets. For example, placing feet correctly on uneven ground or making a character's hand reach for an object, even if the base animation didn't account for it.

Inverse Kinematic (IK)

What it is: Instead of following the animation exactly, Unity adjusts body parts (like hands, feet, or head) in real time to match the environment.
Example in Unity: ○ Feet IK: character's feet align to the slope of the ground. ○ Hand IK: character's hand reaches a door handle even if the animation wasn't made for that exact door.
Why it's useful: Adds realism and responsiveness without needing a custom animation for every situation.
Where it's used: Aiming weapons, opening doors in Outlast ,Synching the arms to the hammer position in getting over it , picking up objects, climbing, or interacting with uneven terrain

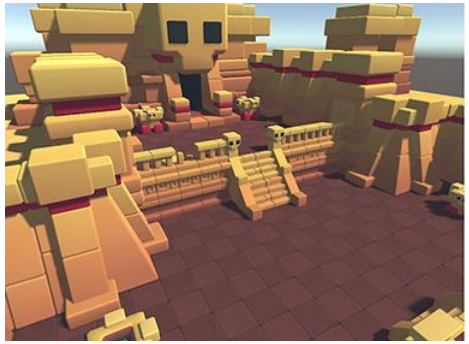
Game Perspective and Types of camera

Orthographic cameras

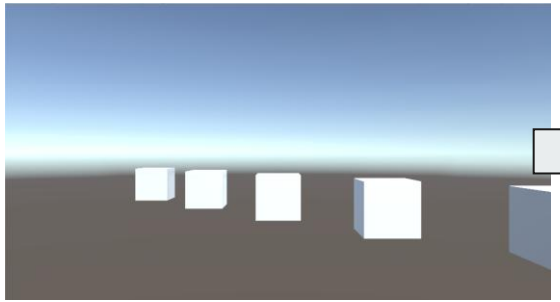
Orthographic cameras do not use perspective or foreshortening.

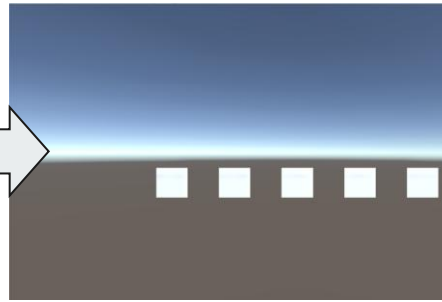
An object viewed in an Orthographic Camera will always appear the same size.

A two-unit-high object will appear twice as tall as a one-unit-high object, regardless of where they are in relation to each other and to the Camera. Parallel lines and edges remain parallel.





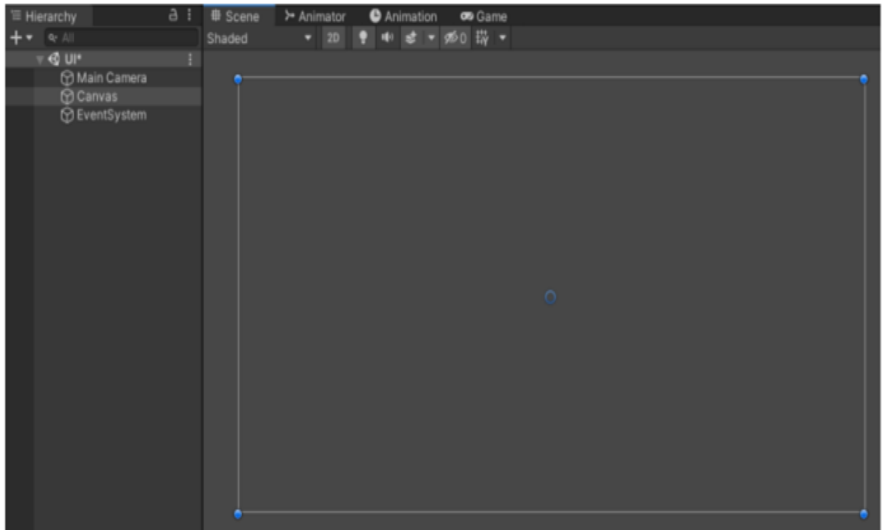




Introduction to UI.

Canvas

- The Canvas is the area that all UI elements should be inside.
  - Canvas is a Game Object with a Canvas component on it, and all UI elements must be children of a Canvas.
  - Creating a new UI element, automatically creates a Canvas.



Unity UI – Visual Components

- **Text:**
  - The Text component has a text area for entering the text that will be displayed.
  - It is possible to set the font, font style, font size, etc.



- **Image:**
  - A sprite can be applied to the Image component under the Source Image property.



Interaction Components

Button:

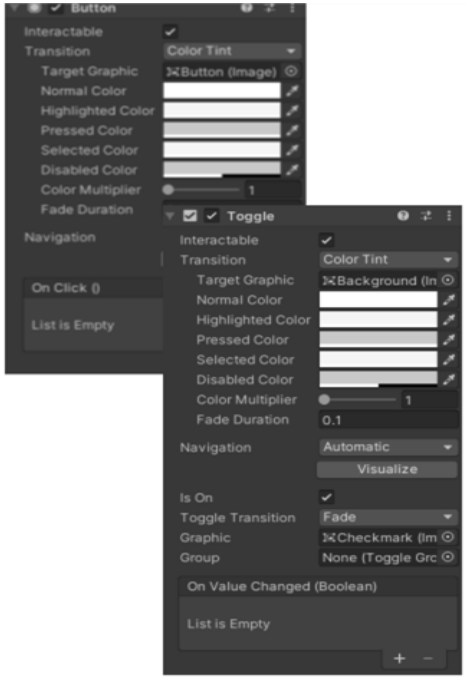
The Button control responds to a click from the user and is used to initiate or confirm an action.

Button

Toggle:

The Toggle control is a checkbox that allows the user to switch an option on or off.

Toggle



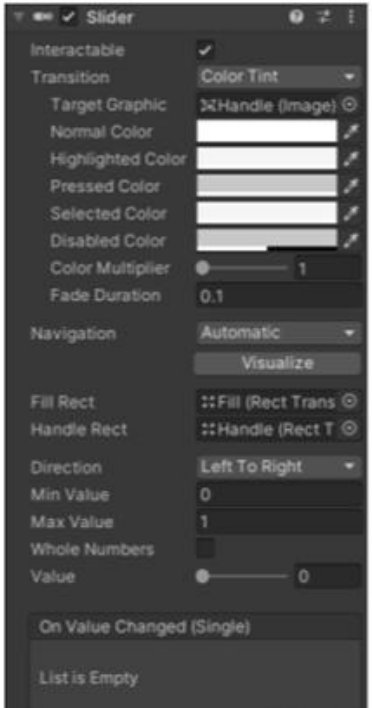
Interaction Components

- Toggle Group:
  - A Toggle Group is used to modify the behavior of a set of Toggles.
  - Pressing a Toggle of a Group to switch it on automatically switches the others off.



- Slider:
  - The Slider control allows the user to select a numeric value from a predetermined range by dragging the mouse.





Interaction Components



- Scrollbar:
  - A Scrollbar has a decimal number Value between 0 and 1.
  - Scrollbars are often used together with a Scroll Rect and a Mask to create a scroll view.

- Dropdown:
  - A Dropdown has a list of options to choose from.





Interaction Components

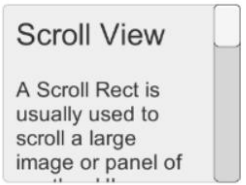
Input Field:

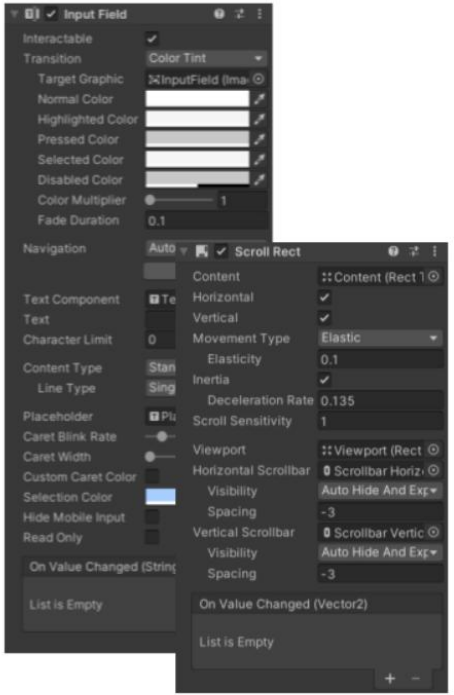
- Used to make the text of a Text Element editable



Scroll Rect (Scroll View):

- Used when the content takes up a lot of space and needs to be displayed in a smallarea. The Scroll Rect allows users to scroll over the content





Events

Unity includes an event system that communicates user interactions to visual elements.
There are two ways to make a visual element
react to an event:

- By implementing and registering a function through the inspector;
- By implementing and registering a function through code;



Events

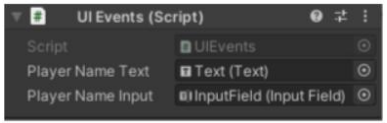
Example 1: registering the function through the inspector

- Event implementation

```
public class UIEvents : MonoBehaviour
{
    public Text playerNameText;
    public InputField playerNameInput;

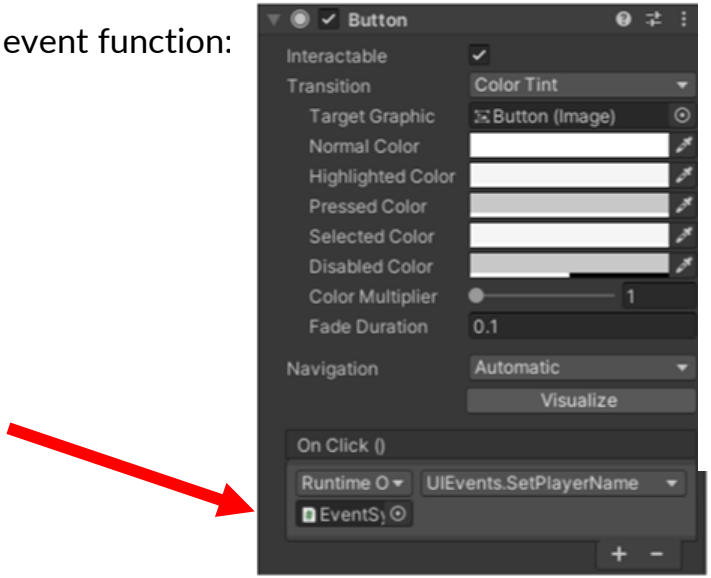
    public void SetPlayerName()
    {
        playerNameText.text = "Current Player: " + playerNameInput.text;
    }
}
```

Add the script
to a GameObject (e.g. EventSystem) and set the properties:



Events

- Select the object and the event function:





Implementing and registering a function through code.

```
public class UIEvents2 : MonoBehaviour
{
    public Text playerNameText;
    public InputField playerNameInput;

    void Start()
    {
        Button button = GetComponent<Button>();
        button.onClick.AddListener(SetPlayerName);
    }

    public void SetPlayerName()
    {
        playerNameText.text = "Current Player: " + playerNameInput.text;
    }
}
```

Instantiate UI Components by Code

- All UI components can be instantiated by code.

```
public class InstantiateUI : MonoBehaviour
{
    public Canvas canvas;
    public Font font;

    void Start()
    {
        GameObject textObj = new GameObject();
        textObj.name = "TestText";
        textObj.transform.SetParent(canvas.transform);
        Text text = textObj.AddComponent<Text>();
        text.text = "My Text";
        text.font = font;
        text.fontStyle = FontStyle.Bold;
        text.rectTransform.anchoredPosition = new Vector3(0, 0);
    }
}
```

Instantiate a Prefab UI

```
public class InstantiateUI2 : MonoBehaviour
{
    public Canvas canvas;
    public Font font;
    public GameObject optionsUIPrefab;

    void Start()
    {
        GameObject optionsUI = GameObject.Instantiate(optionsUIPrefab,
                                                      canvas.transform, false);
        optionsUI.GetComponent<RectTransform>().anchoredPosition = new
                                                              Vector3(0, 0, 0);
    }
}
```



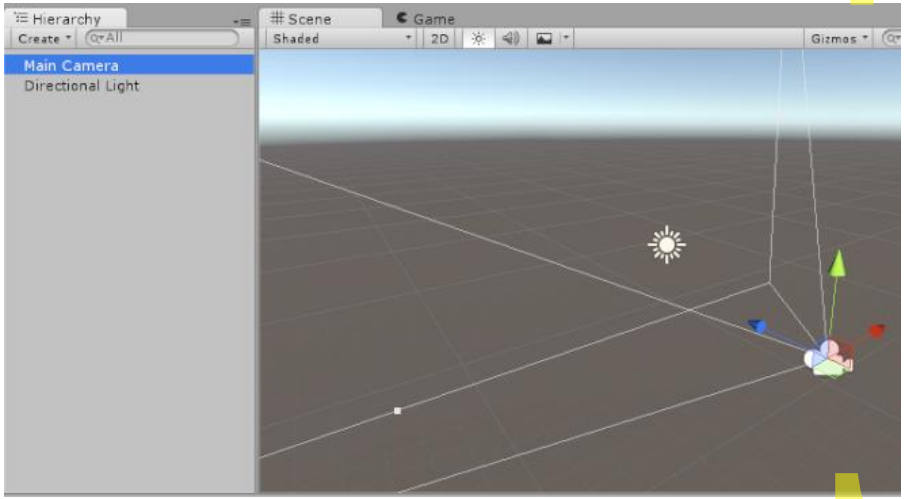
Multiple scene

Scenes

Scenes are where you work with content in Unity.

They are assets that contain all or part of a game or application. For example, you might build a simple game in a single **scene**, while for a more complex game, you might use one scene per level, each with its own environments, characters, obstacles, decorations, and **UI** .

You can create any number of scenes in a project. When you create a new project and open it for the first time, Unity opens a sample scene that contains only a **Camera** and a Light.



Scene Templates

Unity uses scene templates to create new scenes. Scene templates are assets that are stored in a project.

They are similar to scenes, but are designed to be copied rather than used directly.

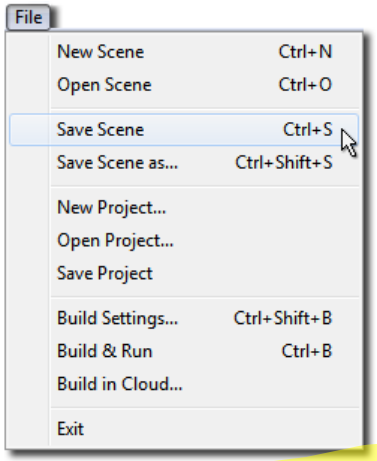
To create new scenes, Unity copies **scene** templates. Think of a scene template as a pre-configured scene that contains all of the content you want to start with.

For example, the default Basic template usually contains a **Camera** and a light.

Saving a scene

Unity stores lots of different types of information about your project, and some of these are saved in different ways to others. This means that *when* your work is saved depends on what kind of changes you are making.

Saving changes to the current scene (“Save Scene”)



Saving your scene

Scene changes include modifications to any objects in the Hierarchy. For example, adding, moving or deleting Game Objects, changing parameters of hierarchy Game Objects in the inspector.

To save changes to the scene, select Save Scene from the file menu, or hit **Ctrl/Cmd + S**. This saves current changes to the scene *and* Does a "**Save Project**" (below).

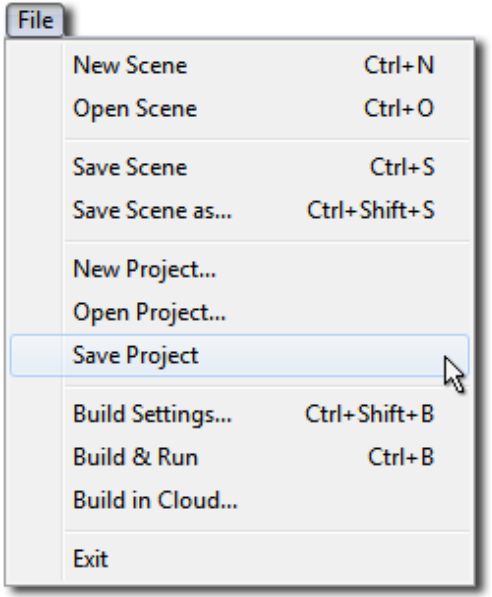
This means that when you do a "Save Scene", *everything* is saved.

Saving project-wide changes (“Save Project”)

Some changes that you can make in Unity are not scene-specific, they are *project-wide*. These settings can be saved independently of the scene changes, by selecting “Save Project” from the file menu.

Using “Save Project” does not save changes to your Scene, only the project-wide changes are saved. You may want to, for instance, save your project but not changes to your scene if you have used a temporary scene to make some changes to a prefab.

The project-wide changes which are saved when you “Save Project” include:



How to load a Scene in Unity

Next, you'll need to add the **Scene Management namespace** to any script that you want to load Scenes from.

Simply add "**using UnityEngine.SceneManagement;**" with the other using directives.

Loading a scene : Using Scene Management

This will allow you to use functions from the **SceneManager class.**

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.SceneManagement;
```

Finally, to load a new Scene, call the **Load Scene** function, passing in the name of the Scene you'd like to load.

Like this :

```
SceneManager.LoadScene("DemoScene");
```

Loading scene with an Index.

Alternatively, you can use the Scene's index instead.

Like this : `SceneManager.LoadScene(1);`

If you don't know the index of the Scene you want to load, you can look it up by opening the Build Settings. You can also reorder Scenes to change their index number.

Loading by a Scene's index can be especially useful if you want to load Scenes in order; For example, if you want the player to proceed to the next Scene in the list at the end of each level.

How to load the next Scene in Unity

To load Scenes in order, i.e. load whatever Scene is next on the list after the current Scene, all you need to do is get the current Scene index and add 1.

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.SceneManagement;
public class LoadScene: MonoBehaviour
{
    public void LoadNextScene()
    {

SceneManager.LoadScene(SceneManager.GetActiv
eScene().buildIndex + 1);
    }

}
```

The two types of loading

There are two different methods available for Loading a scene.

Load Scene and **Load Scene Async**.

Load scene

Load Scene, as used in the earlier example, loads the Scene directly with loading taking place during the next frame

While the Scene loads, the game will freeze.

Music and audio will continue to play but the game will be unresponsive until the load has finished.

Depending on the amount of time that's required to load the next Scene, this pause can give the impression that the game has crashed, unless you hide the pause with a Loading Screen (more on that later).

Load Scene Async

Load Scene Async works in a similar way to Load Scene, except the loading takes place as a background operation and is spread over multiple frames:

```
    SceneManager.LoadSceneAsync(sceneToLoad);
```

There are a few reasons that you might want to do this:

For example, you might want to keep the current Scene running until the new Scene is ready.Or, if your loading screen includes animated elements, such as tooltips or a loading icon, these can only work when loading the Scene in the background.

LoadSceneMode

Use LoadSceneMode to choose what type of Scene loads when using SceneManager.LoadScene.
The available modes are **Single** and **Additive**.

Single mode: loads a standard Unity Scene which then appears on its own in the Hierarchy window.

Additive loads :a Scene which appears in the Hierarchy window while another is active.

Keeping game object between scenes

The load of a new Scene destroys all current Scene objects. Call Object.DontDestroyOnLoad to preserve an Object during scene loading. If the target Object is a component or GameObject, Unity also preserves all of the Transform's children. Object.DontDestroyOnLoad only works for root GameObjects or components on root GameObjects. Object.DontDestroyOnLoad does not return a value.

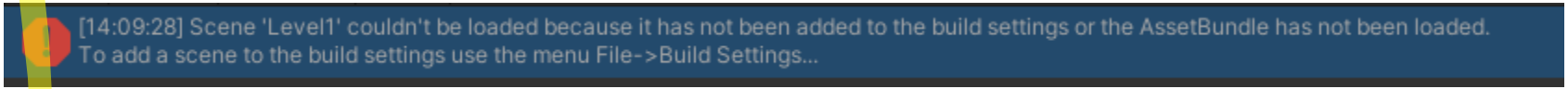
```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class DontDestroy : MonoBehaviour
{
    void Awake()
    {
        GameObject[] objs =
GameObject.FindGameObjectsWithTag("music");

        if (objs.Length > 1)
        {
            Destroy(this.gameObject);
```

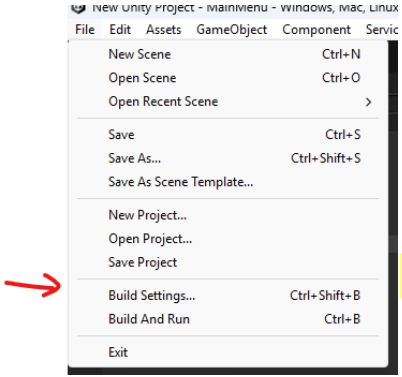
Build settings

Adding a scene to the Build order



Use the **Scene List** to organize the scenes in your build. Unity builds scenes in the order they appear in the list. You can add, exclude, remove, and reorder scenes in the list.

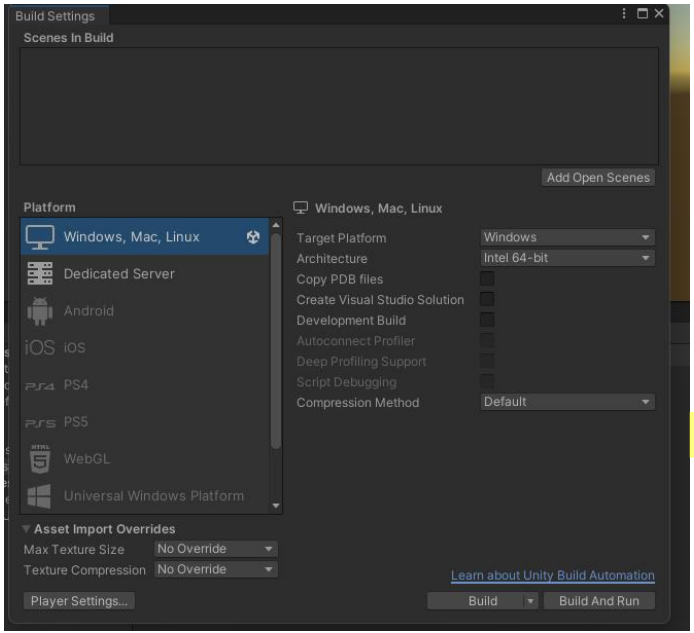
Access the scene list by navigating to **File** > **Build Settings**



Build settings

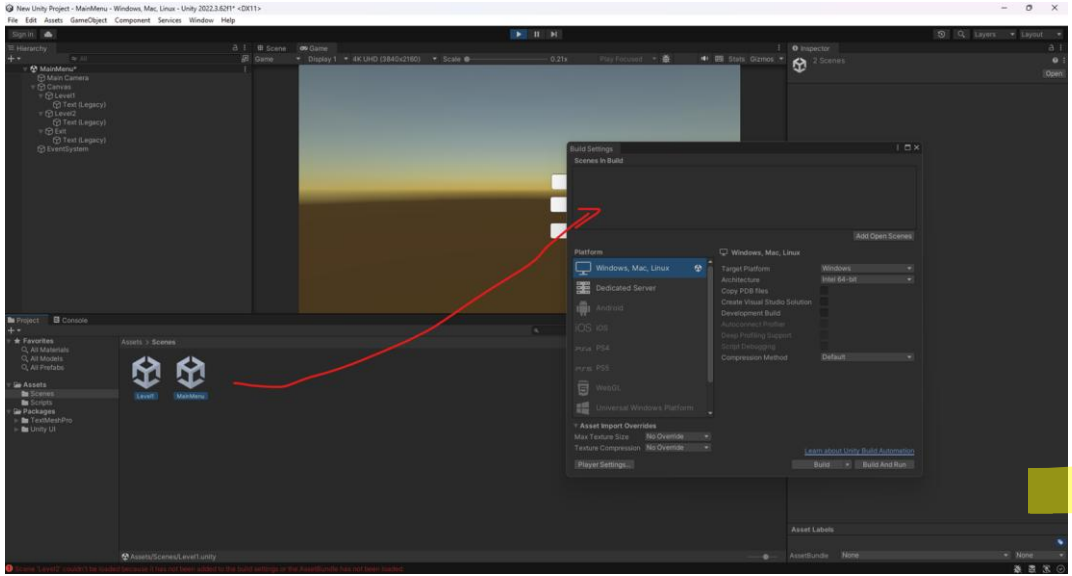
This windows allows you to specify what scenes are part of your game and which are not.

It also allows you to create a standalone executable for your game.



How to add scene to the build settings

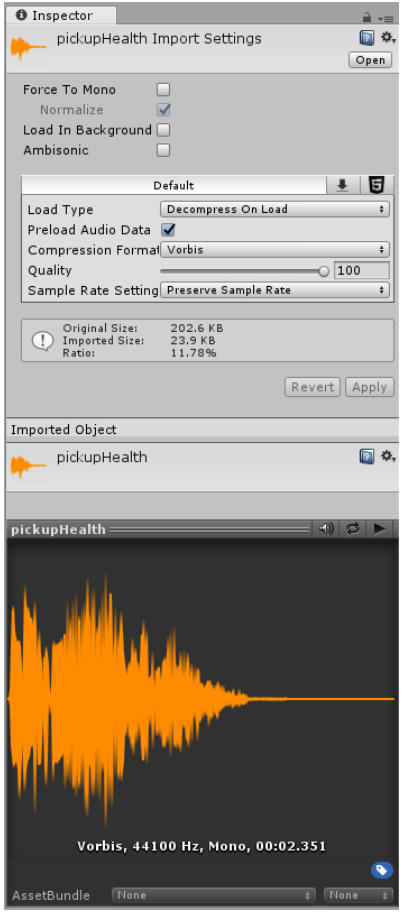
Simply drag and drop the scene you want in your game and drop them in the **Scene in Build** panel



Audio

Audio clip

Audio clips contain the audio data used by Audio Sources. Unity supports mono, stereo, and multichannel audio assets up to eight channels. You can import the following audio file formats in Unity: .aif, .wav, .mp3, and .ogg. Unity also supports importing of tracker modules in the .xm, .mod, .it, and .s3m formats.



Audio Listener

The **Audio Listener** acts as a microphone-like device. It receives input from any given Audio Source in the **scene** and plays sounds through the computer speakers.

For most applications it makes the most sense to attach the listener to the Main Camera. If an audio listener is within the boundaries of a Reverb Zone reverberation is applied to all audible sounds in the scene. Furthermore, Audio Effects can be applied to the listener and it will be applied to all audible sounds in the scene.

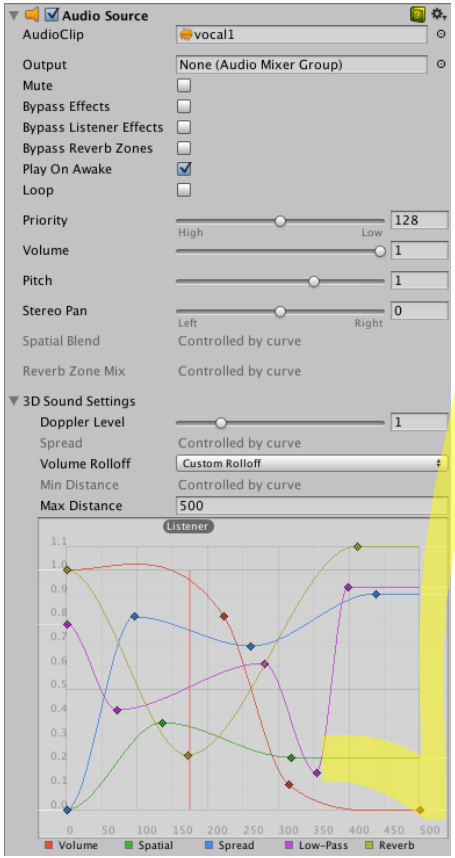


The Audio Listener has no properties. It simply must be added to work. It is always added to the Main Camera by default.

Audio Source

The **Audio Source** plays back an Audio Clip in the **scene**. The clip can be played to an audio listener or through an audio mixer. The audio source can play any type of Audio Clip and can be configured to play these as 2D, 3D, or as a mixture (*SpatialBlend*).

Also, if the listener is within one or multiple Reverb Zones, reverberation is applied to the source. Individual filters can be applied to each audio source for an even richer audio experience.



Playing Audio in Unity

```
AudioSource audioData;

void Start()
{
    audioData = GetComponent<AudioSource>();
    audioData.Play(0);
    Debug.Log("started");
}
```

Particle system

Unity Particle System

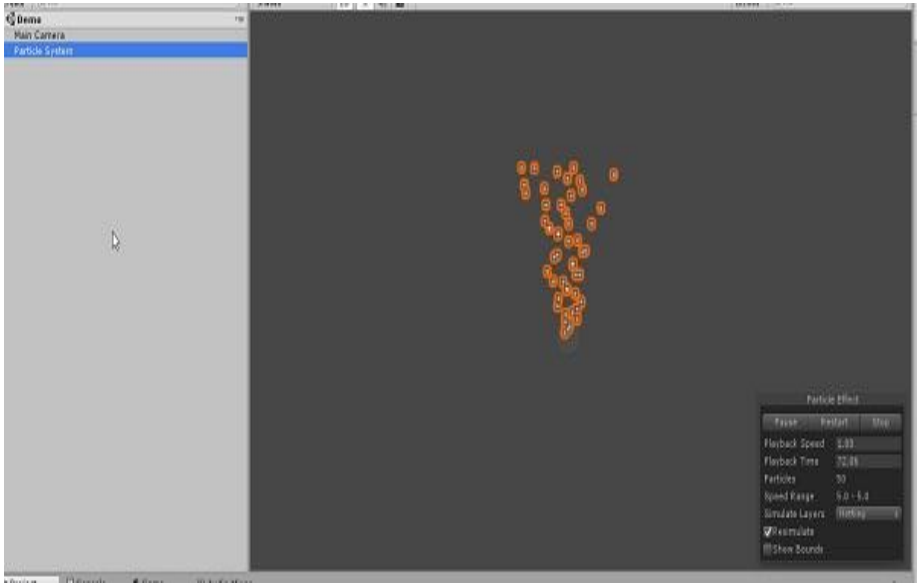
Particle Systems help in generating a large number of particles with small lifespans in an efficient manner. These systems undergo a separate rendering process; they can instantiate particles even when there are hundreds or thousands of objects.

A **particle** is any individual texture, material instance or entity **that is generated by the particle system**. These are not necessarily **dots** floating around in space (although they can be!), and they can be used for a ton of different scenarios.

Creating a Particle System

A GameObject manages a Particle System with the Particle System component attached; particle systems do not require any Assets to set up, although they may require different materials depending on the effect you want.

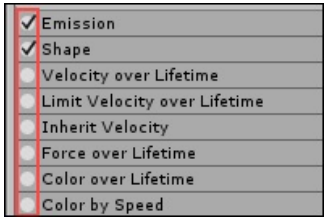
To create a particle system, either add the component Particle System through the **Add Component** setting, or go to the Hierarchy, and select **Create** → **Effects** → **Particle System**. This will generate a new GameObject with the particle system attached.



Particle System Module

If you look at the properties of the Particle System, you will see that comprises many modules. By default, only three modules are active; the **Emission, Shape** and the **Renderer**.

Other modules can be activated by clicking on the small circle next to their name.



Particle System Module : Emission Module

The properties in this module affect the rate and timing of Particle System emissions.

Some properties :

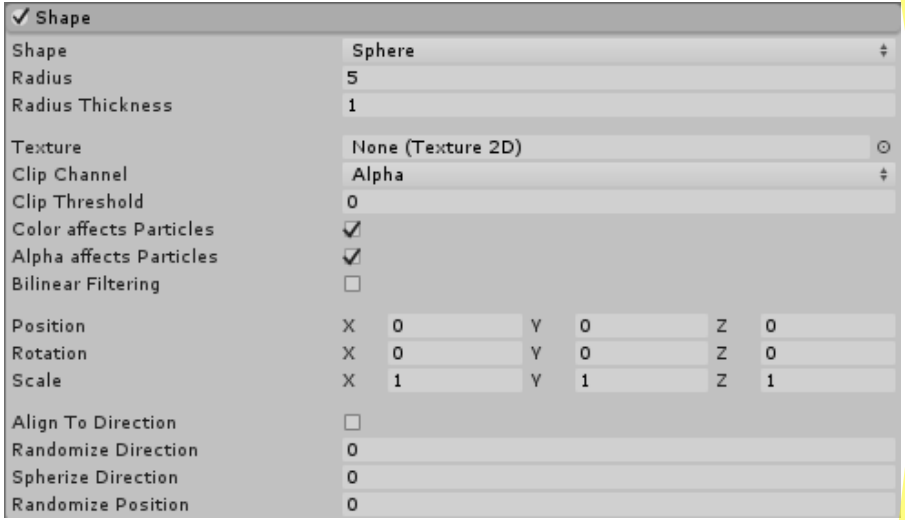
Rate over Time	The number of particles emitted per unit of time.
Rate over Distance	The number of particles emitted per unit of distance moved.
Bursts	A burst is an event which spawns particles. These settings allow particles to be emitted at specified times.

Particle System Module : Shape Module

This module defines the the volume or surface from which particles can be **emitted**, and the direction of the start velocity. The Shape property defines the **shape of the emission volume**, and the rest of the module properties vary depending on the Shape you choose.

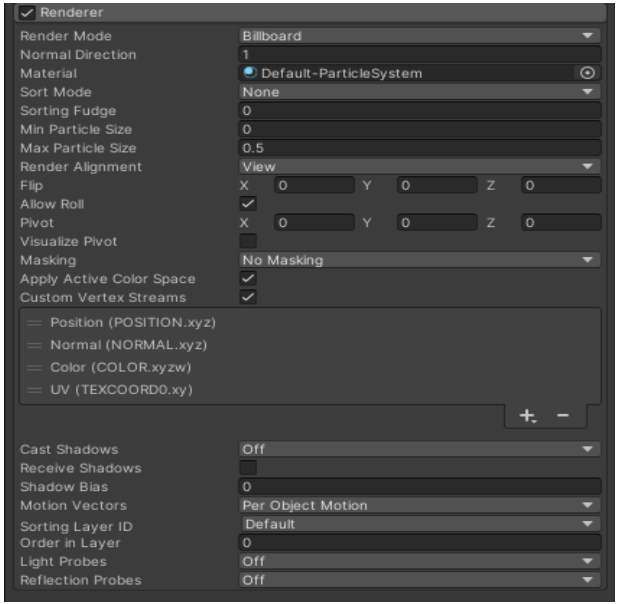
All shapes (except Mesh) have properties that define their dimensions, such as the Radius property. To edit these, drag the handles on the wireframe emitter shape in the Scene view. The choice of shape affects the region from which particles can be launched, but also the initial direction of the particles.

For example, a Sphere emits particles outward in all directions, a Cone emits a diverging stream of particles, and a Mesh emits particles in directions that are normal to the surface.



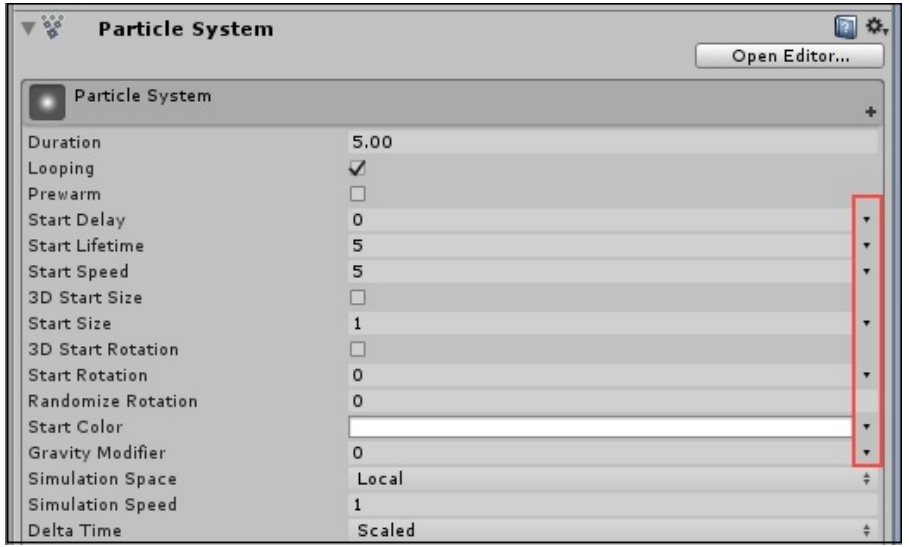
Particle System Module : Renderer module

The Renderer module's settings determine how a particle's image or Mesh is transformed, shaded and overdrawn by other particles.



Particle System properties

To the right of some values, you may notice a small black arrow. This allows you to gain more control over the values of each individual particle. For example, you can set the Start Size to Random between Two Constants to tell the Particle System to render different sized, random particles like a water hose.



Scripting particle systems

With the Particle System in Unity you'll also be able to access, modify, disable Particle Systems via scripts. Much like the Particle System it offers a robust and versatile way to affect Particle Systems through code.

Particle System has been added to a GameObject you can retrieve the Particle System

```
ParticleSystem part = GetComponent<ParticleSystem>();
```

Then you can dig into the ParticleSystem variable to have access to all the other methods and functions available. You can also play or stop the Particle System via code

```
var part = GetComponent<ParticleSystem>();
part.Play();
Destroy(gameObject, part.main.duration);
```



How to do a hitscan ?

What is the Out Keyword in Unity?

The out keyword in Unity is used to return extra information from a function.

Normally, when calling a function, you pass data into it via its parameters and get information back via its return type.

It's likely that you'll have already written functions with a **void** return type, meaning they don't return any information.

Other functions, on the other hand, may specify what type of data is returned from the function when it runs.

```
bool IsAboveTen(float num, out
string result)
{
    if (num > 10)
    {
        result = "The number was
more than 10!";
       return true;
    }
    else
    {
        result = "The number was
not more than 10!";
        return false;
    }

}
```

Raycasting

Raycast

The Raycast function is extremely useful for creating connections between objects in Unity.

For example, if you want to select an object just by looking at it or clicking on it.

Or if you want to shoot an object with a weapon or obstruct a line of sight.

Or perhaps if you need to check how high the player is from the ground, or what it is they're standing on.

That's where Raycasts can help.

What is the Raycast function ?

Raycast in Unity is a **Physics function** that projects a **Ray** into the scene, returning a **boolean value if a target was successfully hit.**

When this happens**, information** about **the hit,** such as **the distance, position or a reference to the object's Transform**, can be stored in a **Raycast Hit variable** for further use.

Raycast are extremely useful for getting information about other objects, performing ground checks or, generally, doing anything that involves a line of sight connection between two objects.

How to create a Ray

A **Ray** is simply a data struct in Unity that represents a point of **origin** and a **direction** for the Ray to travel.



The **origin** is the position in the world that the Ray will start from and takes a Vector3 value.

While the **direction** property takes a trajectory in the form of a normalised Vector3.

Creating a new ray

There are a few ways you can create a new Ray.

```
Ray ray = new Ray(transform.position, transform.forward);
```

Or, alternatively, you can use one of the helper functions to create a Ray automatically, such as a Ray that starts at the

```
Ray ray = Camera.main.ViewportPointToRay(new Vector3 (0.5f, 0.5f, 0));
```

How to visualise Raycast in the scene

The **Debug Class** includes a function, **Draw Ray**, that allows you to render a Ray in the world.

```
Debug.DrawRay(Vector3 origin, Vector3 direction);
```



*Using **Debug.DrawRay**, you can check if your Rays are going where they're meant to.*

How to use the Raycast Hit variable

The Raycast Hit variable is a data struct that stores information about Ray's collision.

It records information like where the hit happened in the world, what object did the Ray collide with and how far away it was from the origin of the Ray.

To use it, all you need to do is declare it:

```
// Container for hit data

   RaycastHit hitData;
```

How to use the Raycast Hit variable :

Then, later, when you use Raycast to fire a Ray into the scene, you'll have a place to store any information retrieved from the hit.

For example, you can get the position of the hit in the world using **RaycastHit.point:**

```
Vector3 hitPosition = hitData.point;
```

Or the distance from the Ray's origin to the point of impact using **RaycastHit.distance:**

```
float hitDistance = hitData.distance;
```

How to use the Raycast Hit variable :

You can also get information about the **Collider** that was hit, such as its **Tag**.

```
// Reads the Collider tag

   string tag = hitData.collider.tag;
```

Or, you could even use Raycast Hit to get a reference to an object's **Transform**:

```
// Gets a Game Object reference from its Transform

   GameObject hitObject = hitData.transform.gameObject;
```

How to use the Raycast function

The Raycast function in Unity allows you to check if a Ray collides with another object in the scene, saving the hit data to a **Raycast Hit** variable if it does.

There are several different versions of Raycast, each from different classes, that can be used in different ways, however, one of the most common ways to use Raycast is using the **Physics Class**, which returns a boolean true or false, depending on if the Ray hit anything.

Example of a using a Raycast Function

```
void FireRay()

{

    Ray ray = new Ray(transform.position, transform.forward);

    RaycastHit hitData;

    Physics.Raycast(ray, out hitData);

}
```

This will fire the ray that you've created into the scene and save information about anything that's hit to the Raycast Hit variable you declared.

This works because of the **Out Keyword** in the function.

Understanding the Physics.Raycast

Raycast takes an **out Raycast Hit** argument, meaning that a **Raycast Hit** variable will need to be specified when you call it.

Then when the function runs, **Raycast Hit data** will be stored in the variable you specified, allowing you to access it. The out keyword allows you to save information in addition to the return type of Raycast.

So what is the return type? Physics.Raycast has a **bool** return type, which returns true if the Ray hits a target.

This means that you can place the Raycast function in an **if condition**.

Using raycast in an If condition

Using Raycast in this way means that code inside the if condition will only run if the Ray actually hits something.

```
void Update()
{
    Ray ray = new Ray(transform.position, transform.forward);
    RaycastHit hitData;
    if (Physics.Raycast(ray, out hitData))
    {
            // The Ray hit something!
    }

}
```

Limit the Raycast range by entering a max distance

Most of the overload methods available for Physics.Raycast also allow you to specify a max distance as a float value.

```
Ray ray = new Ray(transform.position, transform.forward);
if (Physics.Raycast(ray, 10))
{
    // Hit Something closer than 10 units away

}
```

How to hit multiple objects using Raycast

Raycast All works a lot like Raycast, except that instead of hitting and returning only one object it can return multiple objects, storing them in an array.

To use **Raycast All**, you'll need to **declare an array of Raycast Hit** variables and assign them whe

```
public RaycastHit[] hits;
void Update()
{
    Ray ray = new Ray(transform.position, transform.forward);
    hits = Physics.RaycastAll(ray);

}
```

How to hit multiple objects using Raycast

Or you could destroy all of the objects in a Ray's path :

```
public RaycastHit[] hits;
void FireLaser()
{
    Ray ray = new Ray(transform.position, transform.forward);
    hits = Physics.RaycastAll(ray);
    foreach(RaycastHit obj in hits)
    {
        Destroy(obj.transform.gameObject);
    }
}
```